

EDA : all about analyzing the dataset and summarizing the key insights and characteristics of data
 EDA checklist: 1. Understanding the dataset, and its shape 2. checking the data types of each columns 3. categorical and numerical columns 4. checking for missing values 5. Descriptive summary of dataset 6. Groupby for classification problems

```
In [38]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
```

```
In [20]: data = pd.read_csv(r"data.csv")
data.head()
```

```
Out[20]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns



```
In [21]: data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    569 non-null    int64
1   diagnosis                            569 non-null    str
2   radius_mean                          569 non-null    float64
3   texture_mean                         569 non-null    float64
4   perimeter_mean                       569 non-null    float64
5   area_mean                            569 non-null    float64
6   smoothness_mean                      569 non-null    float64
7   compactness_mean                     569 non-null    float64
8   concavity_mean                       569 non-null    float64
9   concave points_mean                  569 non-null    float64
10  symmetry_mean                        569 non-null    float64
11  fractal_dimension_mean               569 non-null    float64
12  radius_se                            569 non-null    float64
13  texture_se                           569 non-null    float64
14  perimeter_se                         569 non-null    float64
15  area_se                              569 non-null    float64
16  smoothness_se                        569 non-null    float64
17  compactness_se                       569 non-null    float64
18  concavity_se                         569 non-null    float64
19  concave points_se                    569 non-null    float64
20  symmetry_se                          569 non-null    float64
21  fractal_dimension_se                 569 non-null    float64
22  radius_worst                         569 non-null    float64
23  texture_worst                        569 non-null    float64
24  perimeter_worst                      569 non-null    float64
25  area_worst                           569 non-null    float64
26  smoothness_worst                     569 non-null    float64
27  compactness_worst                     569 non-null    float64
28  concavity_worst                       569 non-null    float64
29  concave points_worst                  569 non-null    float64
30  symmetry_worst                       569 non-null    float64
31  fractal_dimension_worst               569 non-null    float64
32  Unnamed: 32                           0 non-null      float64
dtypes: float64(31), int64(1), str(1)
memory usage: 147.4 KB
```

```
In [22]: data.drop('Unnamed: 32', axis=1, inplace=True)
```

```
In [23]: data.head(2)
```

Out[23]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness
0	842302	M	17.99	10.38	122.8	1001.0	
1	842517	M	20.57	17.77	132.9	1326.0	

2 rows × 32 columns

```
In [24]: data.shape
```

```
Out[24]: (569, 32)
```

```
In [25]: # remove column 'id'  
data.drop('id', axis=1, inplace=True)
```

```
In [26]: # check for missing values  
data.isna().sum()
```

```
Out[26]: diagnosis          0  
radius_mean              0  
texture_mean            0  
perimeter_mean          0  
area_mean               0  
smoothness_mean         0  
compactness_mean        0  
concavity_mean          0  
concave points_mean     0  
symmetry_mean           0  
fractal_dimension_mean  0  
radius_se               0  
texture_se              0  
perimeter_se            0  
area_se                0  
smoothness_se           0  
compactness_se          0  
concavity_se            0  
concave points_se       0  
symmetry_se             0  
fractal_dimension_se    0  
radius_worst            0  
texture_worst           0  
perimeter_worst         0  
area_worst              0  
smoothness_worst        0  
compactness_worst       0  
concavity_worst         0  
concave points_worst    0  
symmetry_worst          0  
fractal_dimension_worst 0  
dtype: int64
```

```
In [27]: # dataset has no missing values  
# check statistical summary of dataset  
data.describe()
```

Out[27]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	comp
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	14.127292	19.289649	91.969033	654.889104	0.096360	
std	3.524049	4.301036	24.298981	351.914129	0.014064	
min	6.981000	9.710000	43.790000	143.500000	0.052630	
25%	11.700000	16.170000	75.170000	420.300000	0.086370	
50%	13.370000	18.840000	86.240000	551.100000	0.095870	
75%	15.780000	21.800000	104.100000	782.700000	0.105300	
max	28.110000	39.280000	188.500000	2501.000000	0.163400	

8 rows × 30 columns



In [28]: `# check the distribution of target Variable`
`data['diagnosis'].value_counts()`

Out[28]: `diagnosis`
`B 357`
`M 212`
`Name: count, dtype: int64`

In [29]: `# Encoding target variable`
`lbl = LabelEncoder()`
`labels = lbl.fit_transform(data.diagnosis)`
`data['target'] = labels`
`data.head()`

Out[29]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
0	M	17.99	10.38	122.80	1001.0	0.11840
1	M	20.57	17.77	132.90	1326.0	0.08474
2	M	19.69	21.25	130.00	1203.0	0.10960
3	M	11.42	20.38	77.58	386.1	0.14250
4	M	20.29	14.34	135.10	1297.0	0.10030

5 rows × 32 columns



In [30]: `data.target.value_counts()`
`# Benign --> 0`
`# Malignant --> 1`

```
Out[30]: target
0      357
1      212
Name: count, dtype: int64
```

```
In [31]: data.drop('diagnosis', axis=1, inplace=True)
```

```
In [32]: # Grouping the data based on target

data.groupby('target').mean()
```

```
Out[32]:
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean
target						
0	12.146524	17.914762	78.075406	462.790196	0.092478	0.016199
1	17.462830	21.604906	115.365377	978.376415	0.102898	0.027433

target

0	12.146524	17.914762	78.075406	462.790196	0.092478
1	17.462830	21.604906	115.365377	978.376415	0.102898

2 rows × 30 columns



```
In [33]: # we can see that for most of the features, the mean values are higher for Malignant
```

```
In [34]: data.groupby('target').median()
```

```
Out[34]:
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean
target						
0	12.200	17.39	78.18	458.4	0.09076	0.016199
1	17.325	21.46	114.20	932.0	0.10220	0.027433

target

0	12.200	17.39	78.18	458.4	0.09076
1	17.325	21.46	114.20	932.0	0.10220

2 rows × 30 columns



```
In [35]: data.agg(['min', 'max', 'mean', 'median', 'std', 'skew'])
```

Out[35]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	com
min	6.981000	9.710000	43.790000	143.500000	0.052630	
max	28.110000	39.280000	188.500000	2501.000000	0.163400	
mean	14.127292	19.289649	91.969033	654.889104	0.096360	
median	13.370000	18.840000	86.240000	551.100000	0.095870	
std	3.524049	4.301036	24.298981	351.914129	0.014064	
skew	0.942380	0.650450	0.990650	1.645732	0.456324	

6 rows × 31 columns



In [37]:

```
data.groupby('target').agg(['min', 'max', 'count'])
```

Out[37]:

	radius_mean			texture_mean			perimeter_mean			area_mean	...	conc
	min	max	count	min	max	count	min	max	count	min	...	
target												
0	6.981	17.85	357	9.71	33.81	357	43.79	114.6	357	143.5	...	
1	10.950	28.11	212	10.38	39.28	212	71.90	188.5	212	361.6	...	

2 rows × 90 columns



Summary from EDA: 1. No missing values 2. All values are continuous except for the target column 3. Mean is slightly more than median for most of the features, so it is right skewed 4. slight imbalance in dataset. (Benign (0) cases is 357 are more than Malignant(1) cases which is 212 only 5. Mean of most features are clearly larger for Malignant compared to the Benign cases.

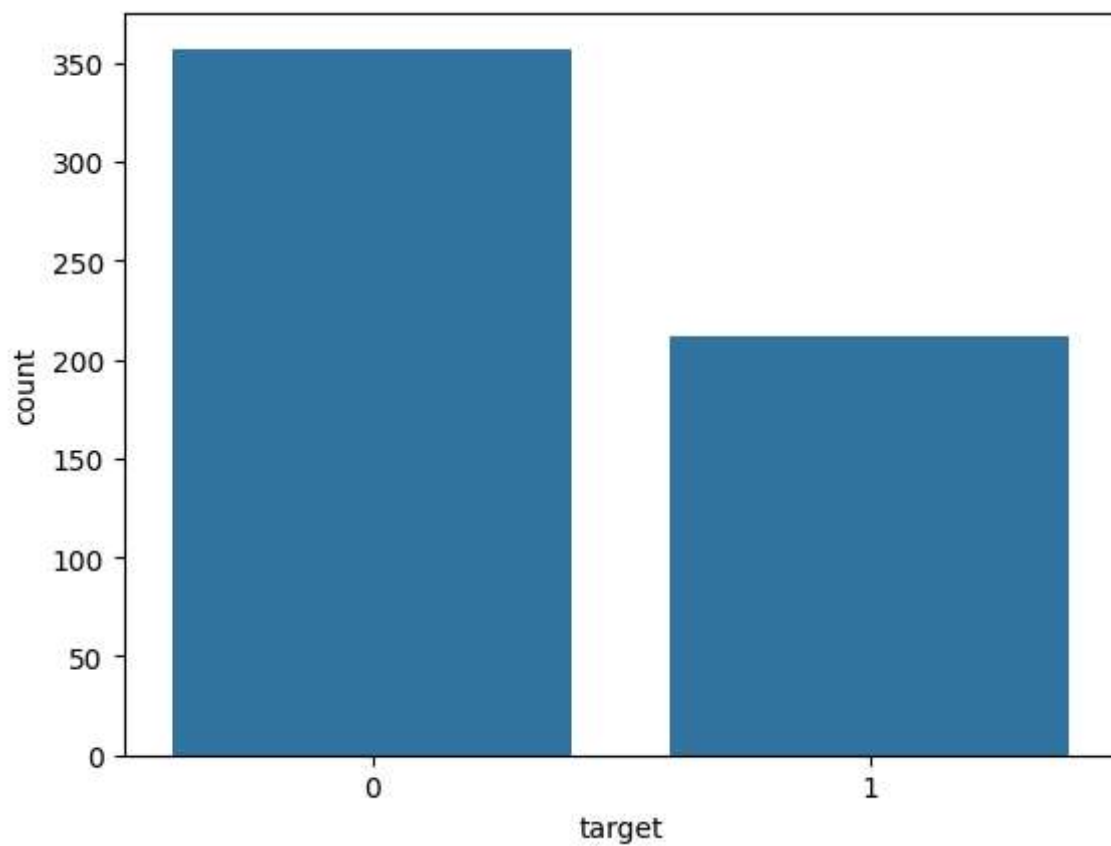
Data Visualization

In [39]:

```
sns.countplot(data=data, x='target')
```

Out[39]:

<Axes: xlabel='target', ylabel='count'>



```
In [40]: # get all the columns of dataframe
for col in data:
    print(col)
```

```

radius_mean
texture_mean
perimeter_mean
area_mean
smoothness_mean
compactness_mean
concavity_mean
concave points_mean
symmetry_mean
fractal_dimension_mean
radius_se
texture_se
perimeter_se
area_se
smoothness_se
compactness_se
concavity_se
concave points_se
symmetry_se
fractal_dimension_se
radius_worst
texture_worst
perimeter_worst
area_worst
smoothness_worst
compactness_worst
concavity_worst
concave points_worst
symmetry_worst
fractal_dimension_worst
target

```

```
In [41]: data.columns
```

```

Out[41]: Index(['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
               'smoothness_mean', 'compactness_mean', 'concavity_mean',
               'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
               'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
               'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
               'fractal_dimension_se', 'radius_worst', 'texture_worst',
               'perimeter_worst', 'area_worst', 'smoothness_worst',
               'compactness_worst', 'concavity_worst', 'concave points_worst',
               'symmetry_worst', 'fractal_dimension_worst', 'target'],
              dtype='str')

```

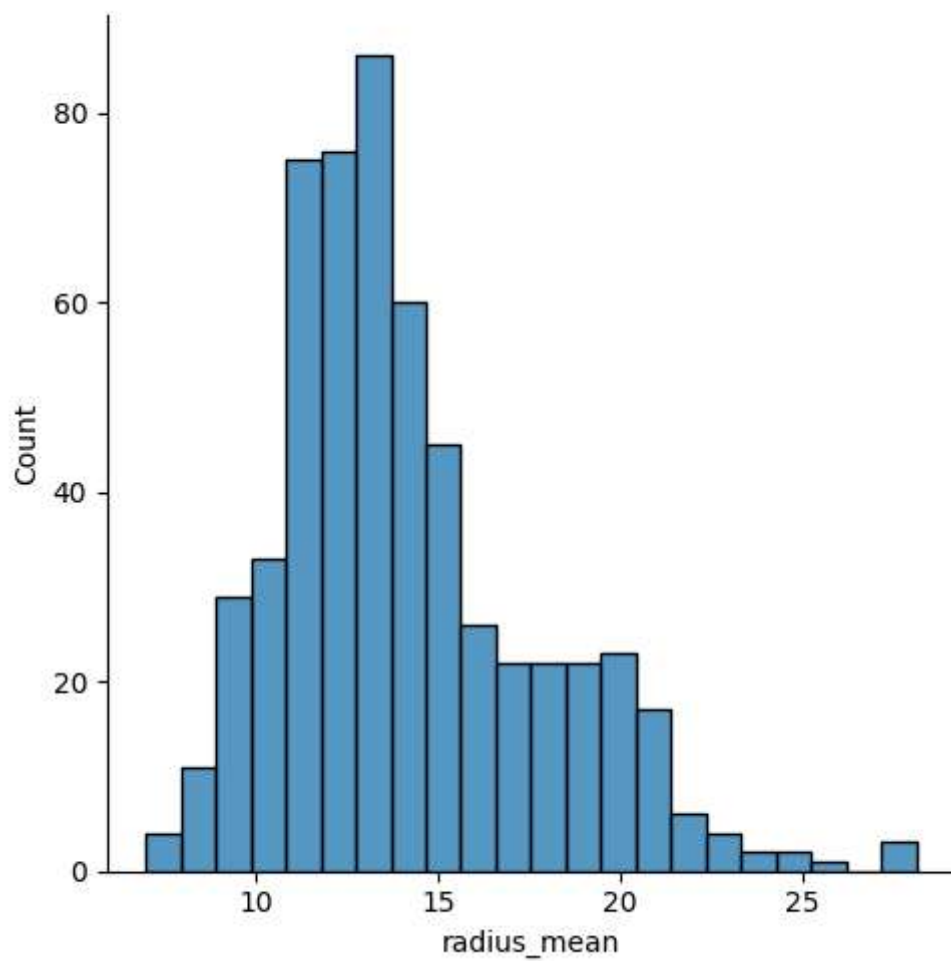
```

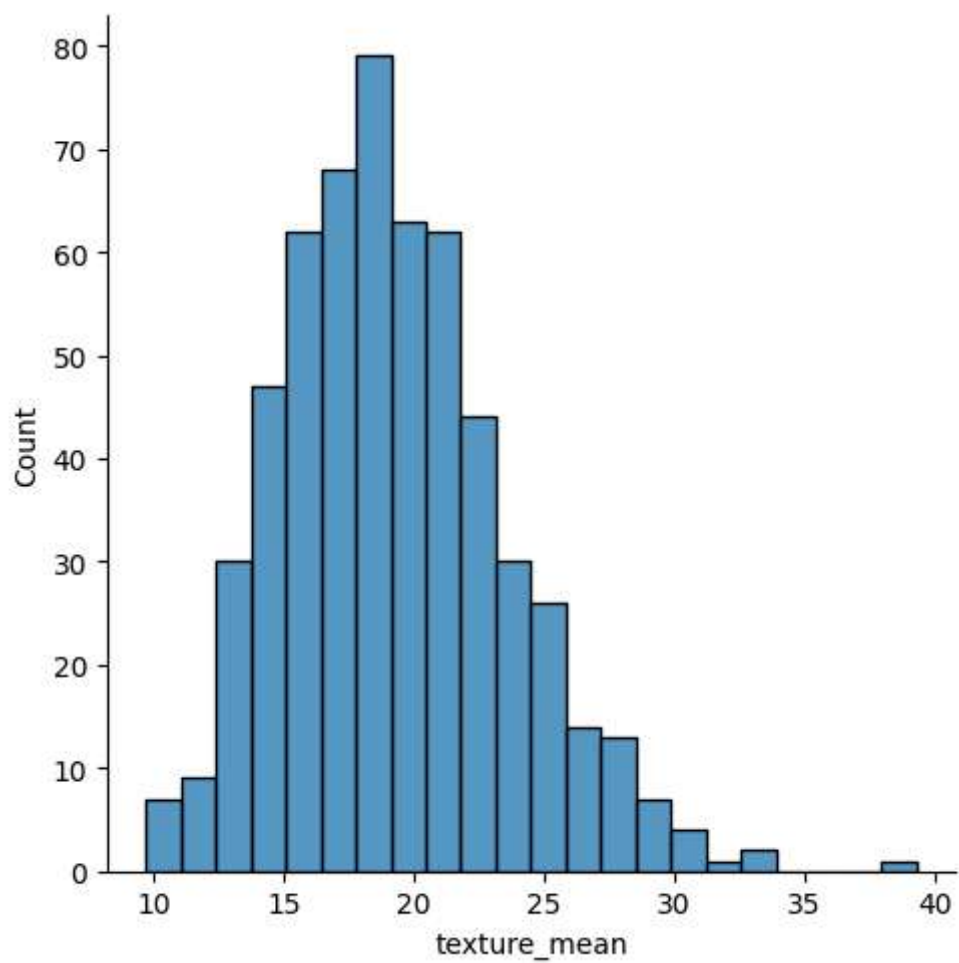
In [42]: # plot distribution plot for all columns
        for col in data.columns:
            sns.displot(x=col, data=data)

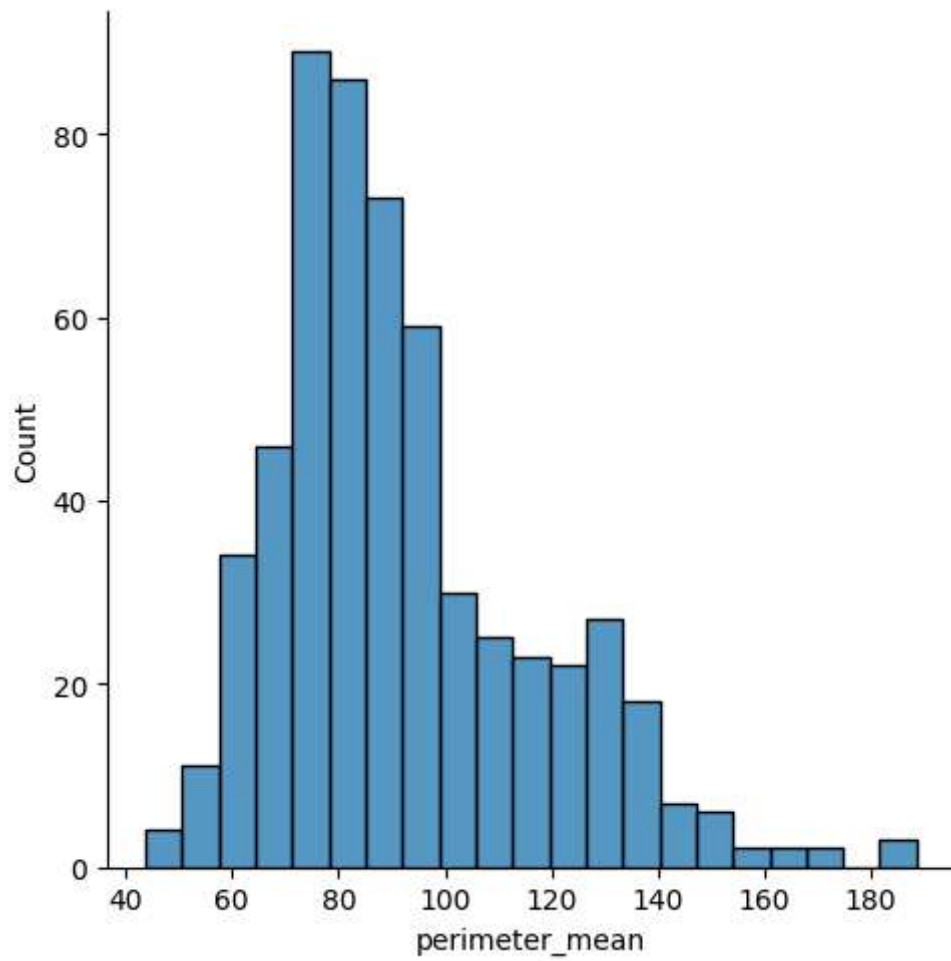
```

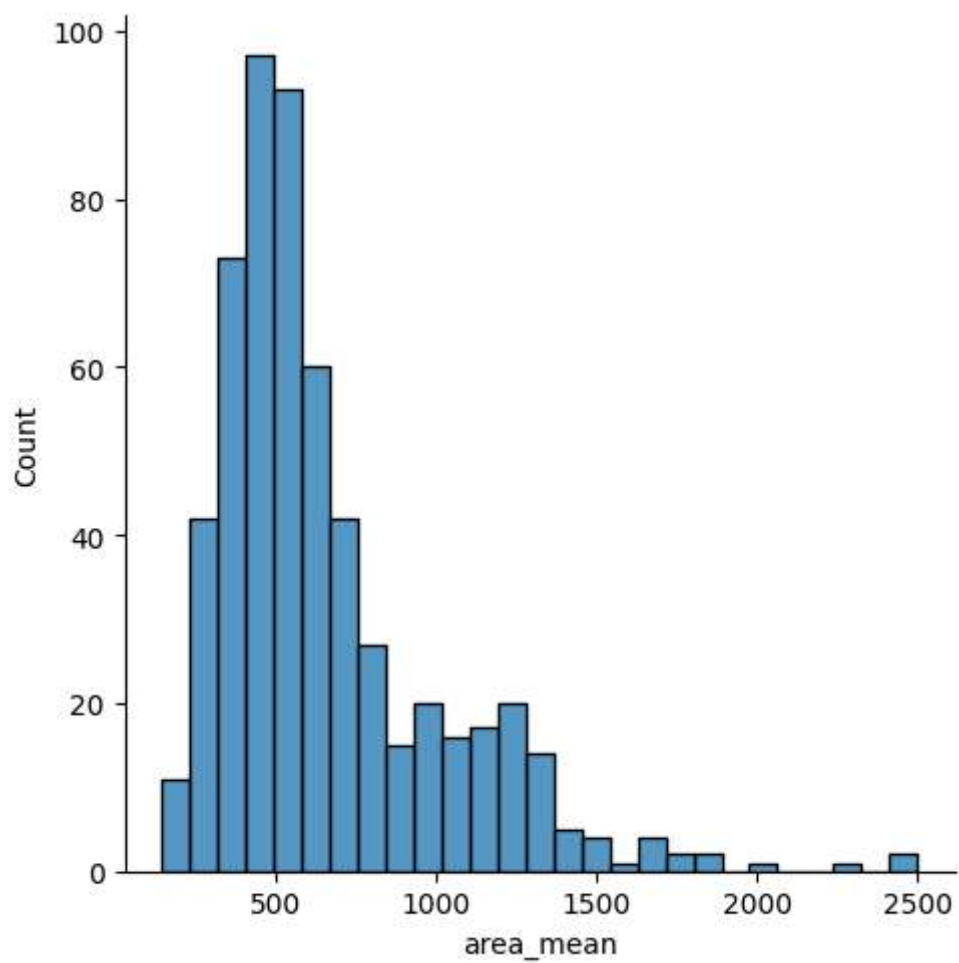
D:\my project\codeit_datascience\data_science\venv\Lib\site-packages\seaborn\axisgrid.py:453: RuntimeWarning: More than 20 figures have been opened. Figures created through the pyplot interface (`matplotlib.pyplot.figure`) are retained until explicitly closed and may consume too much memory. (To control this warning, see the rcParam `figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.

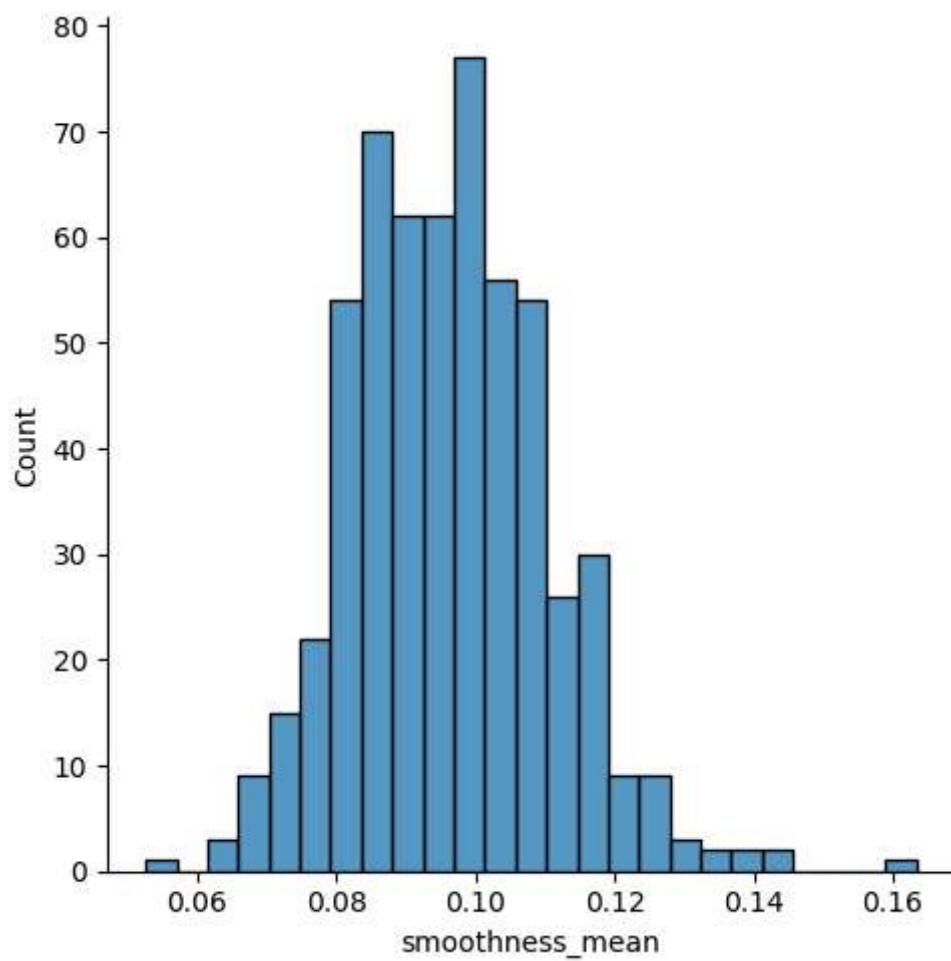
```
fig = plt.figure(figsize=figsize)
```

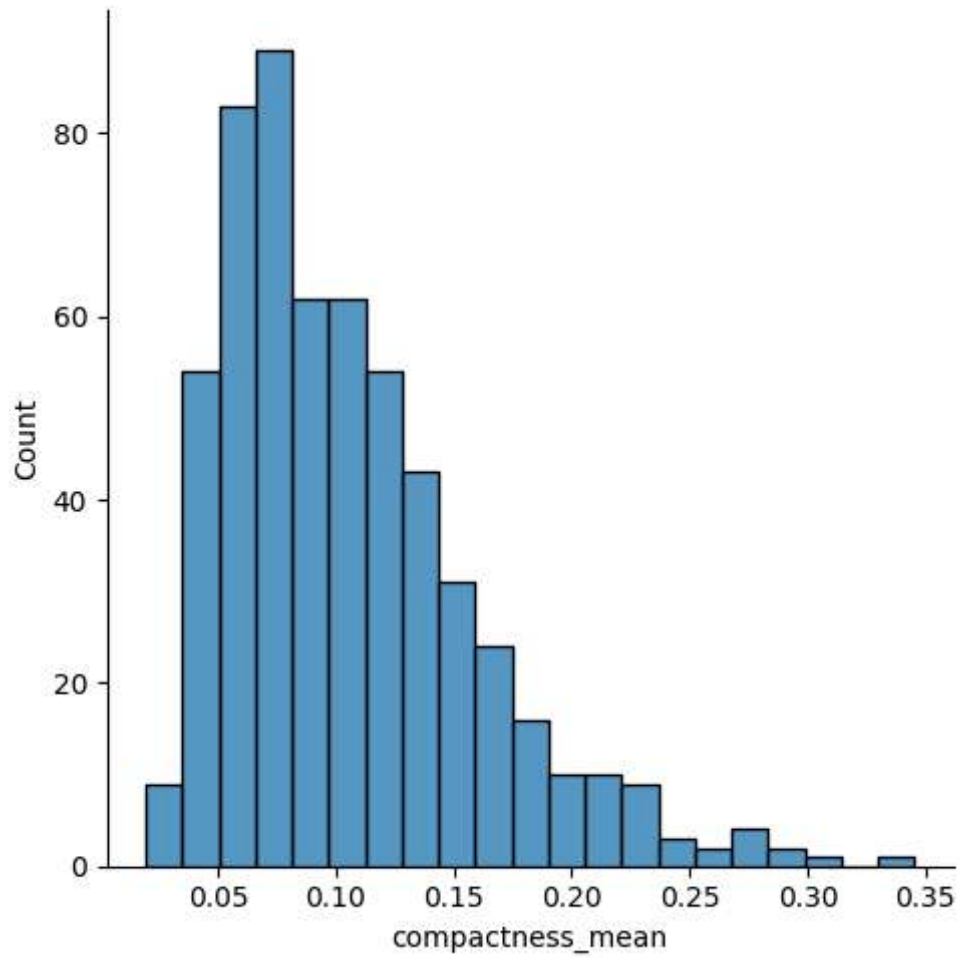


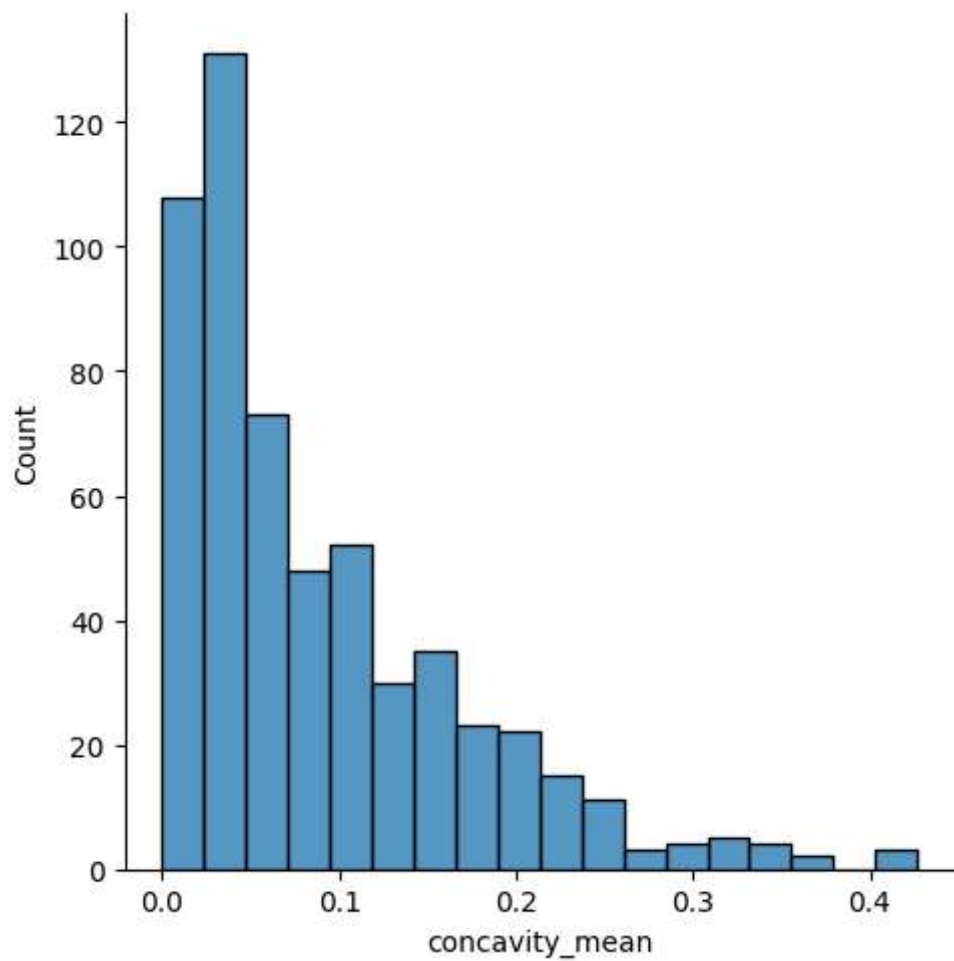


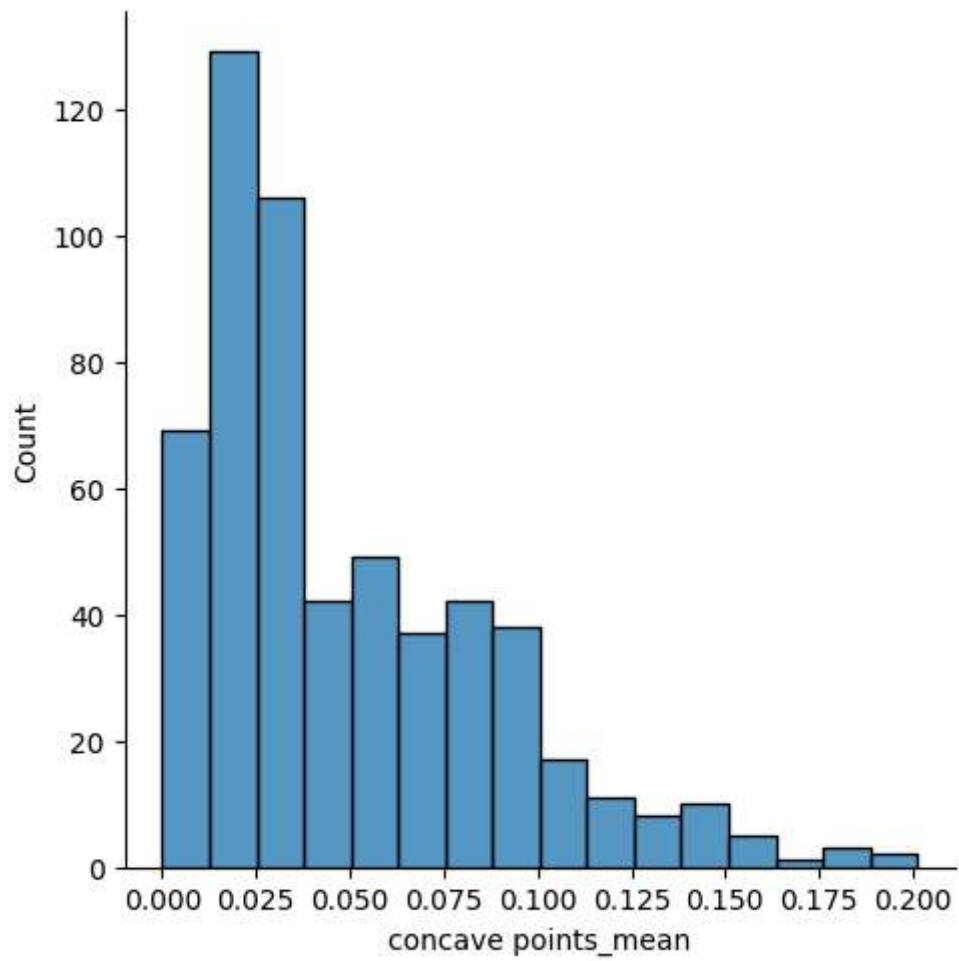


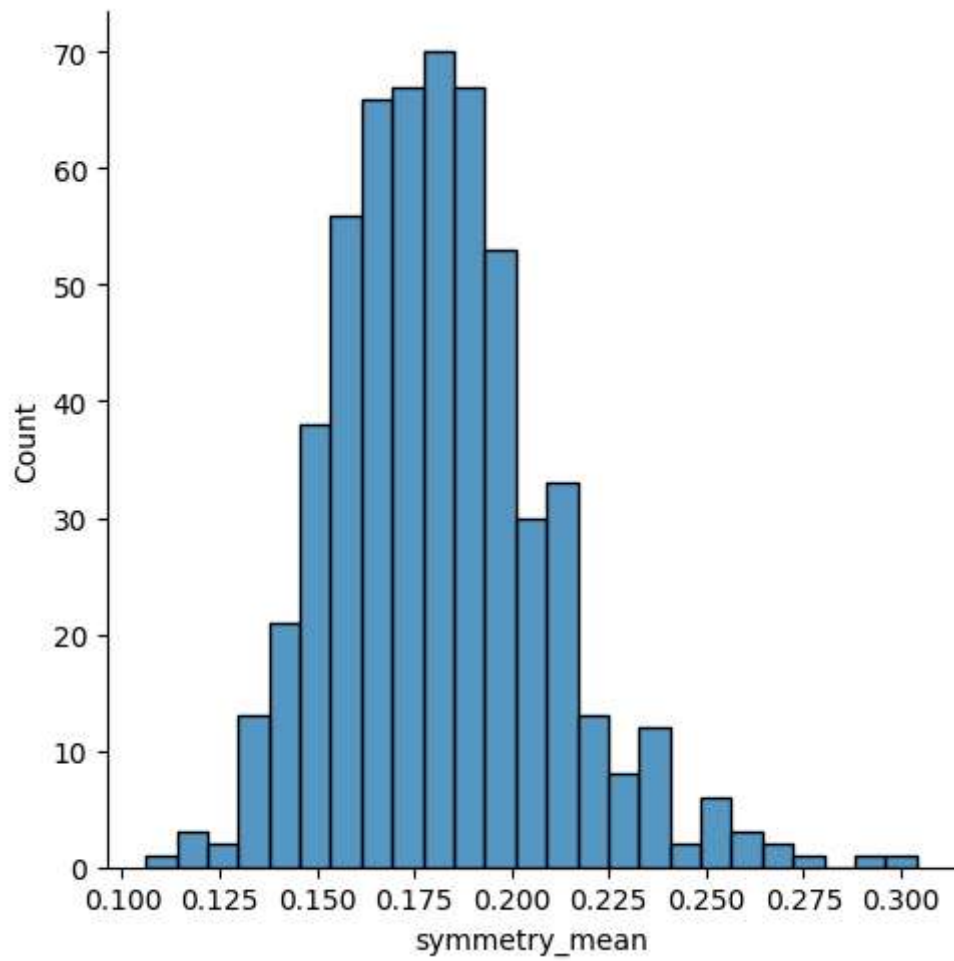


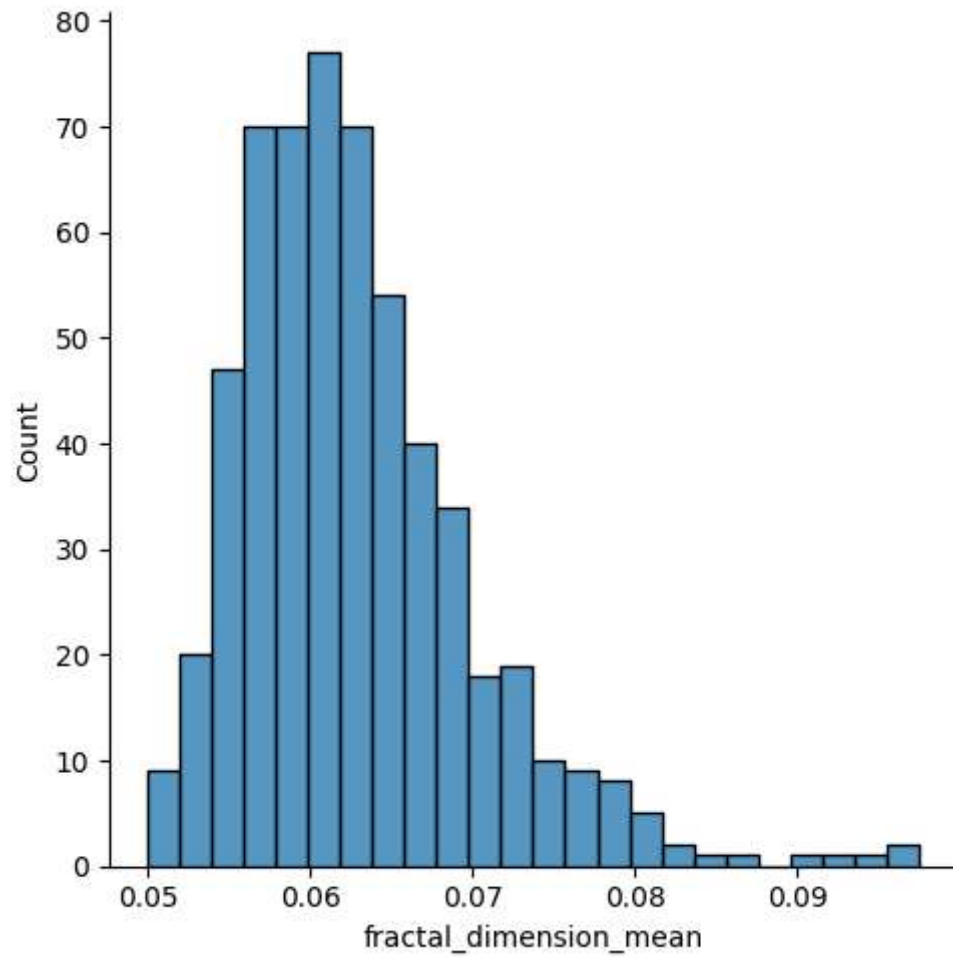


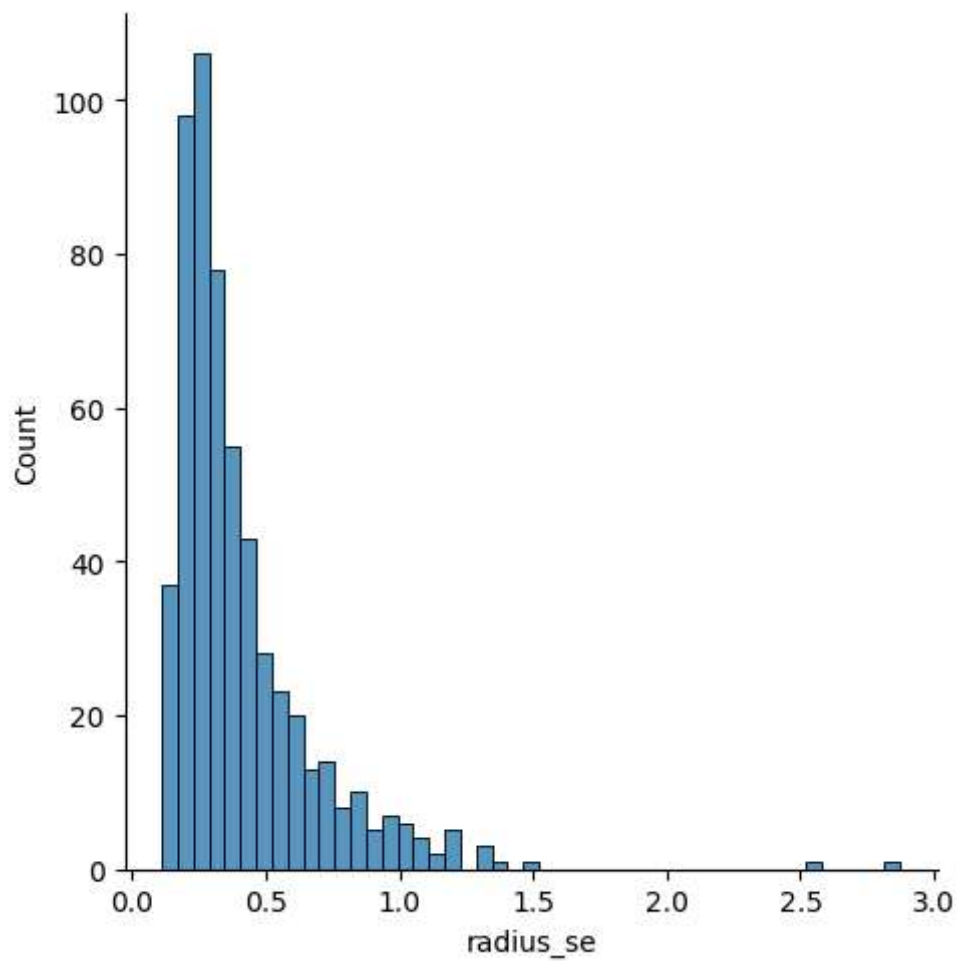


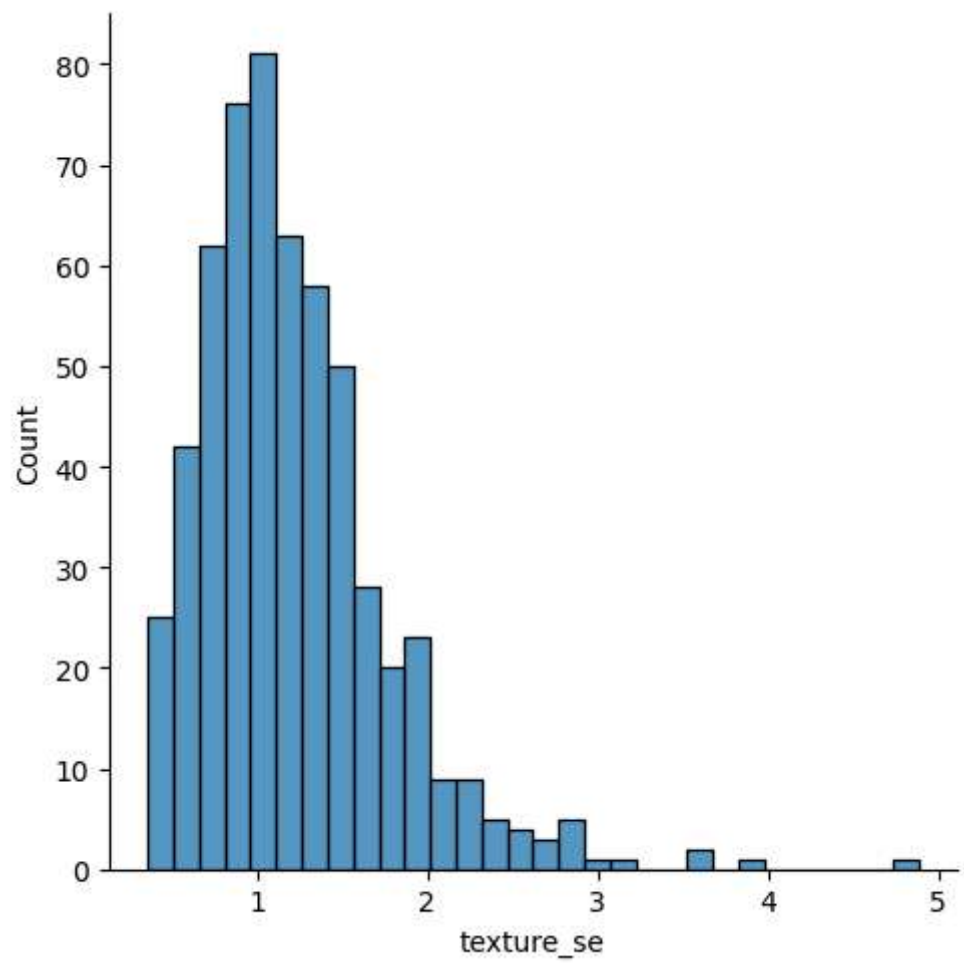


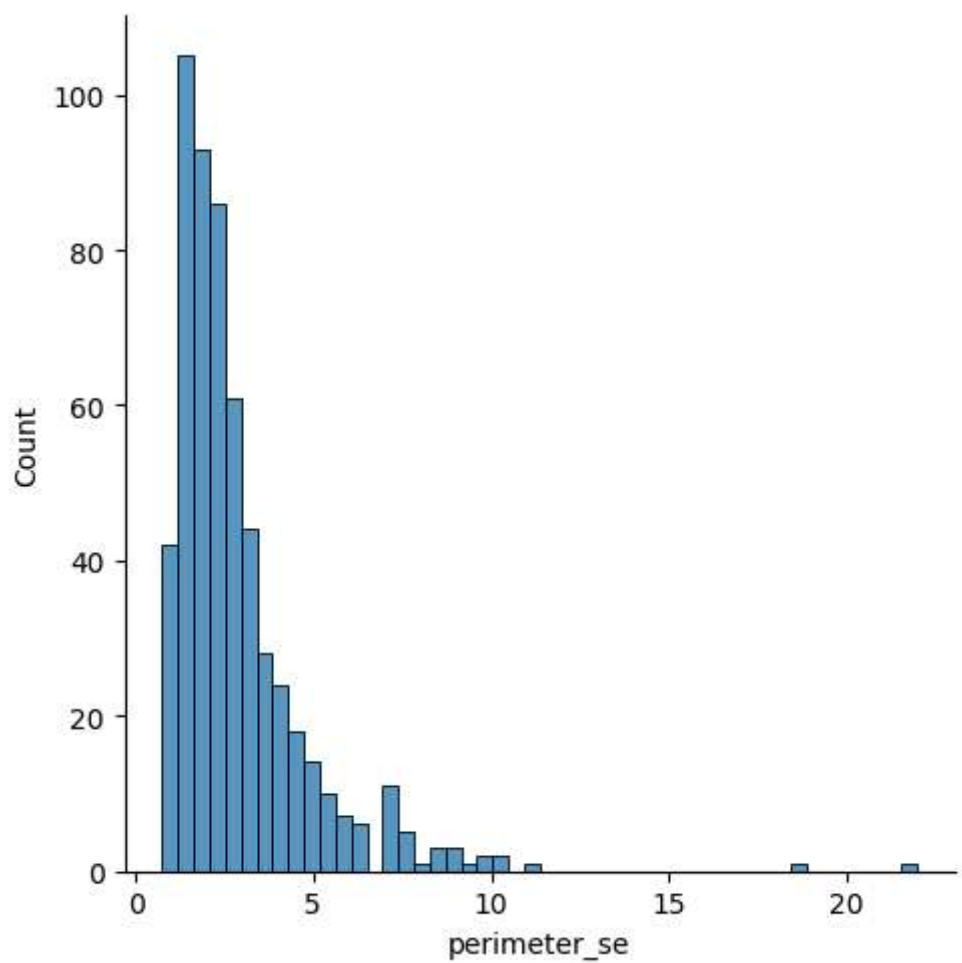


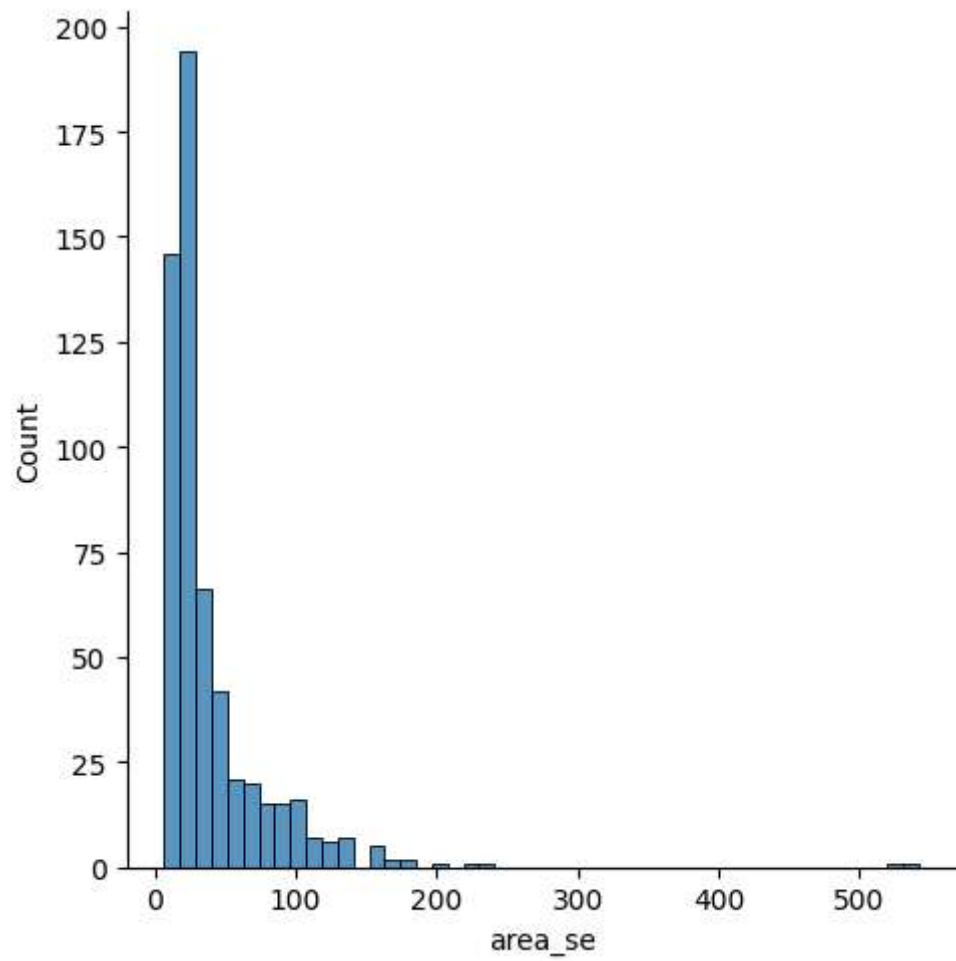


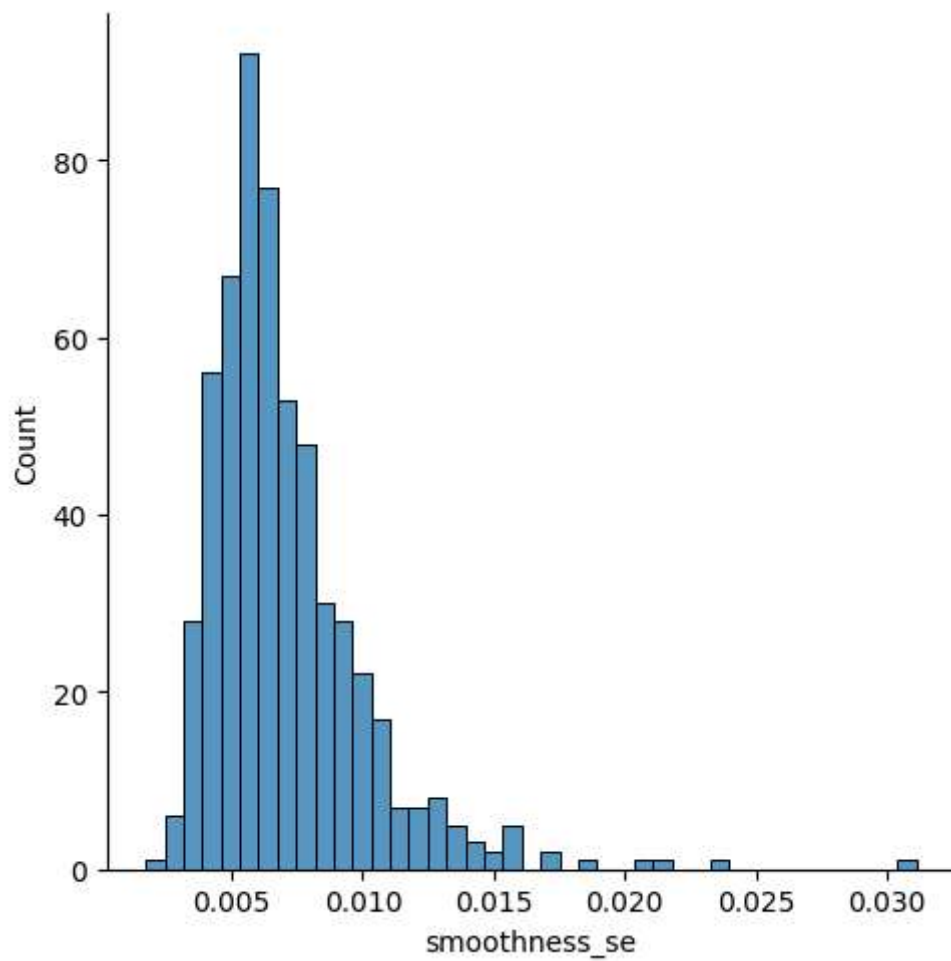


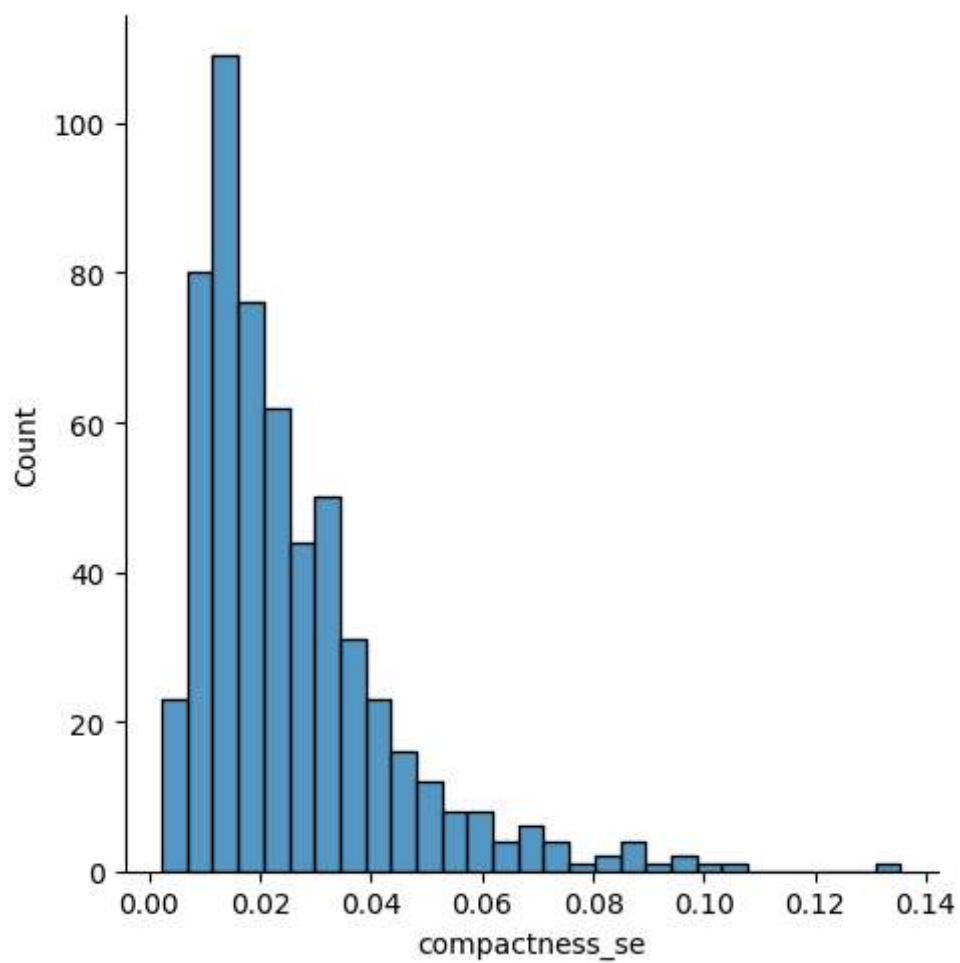


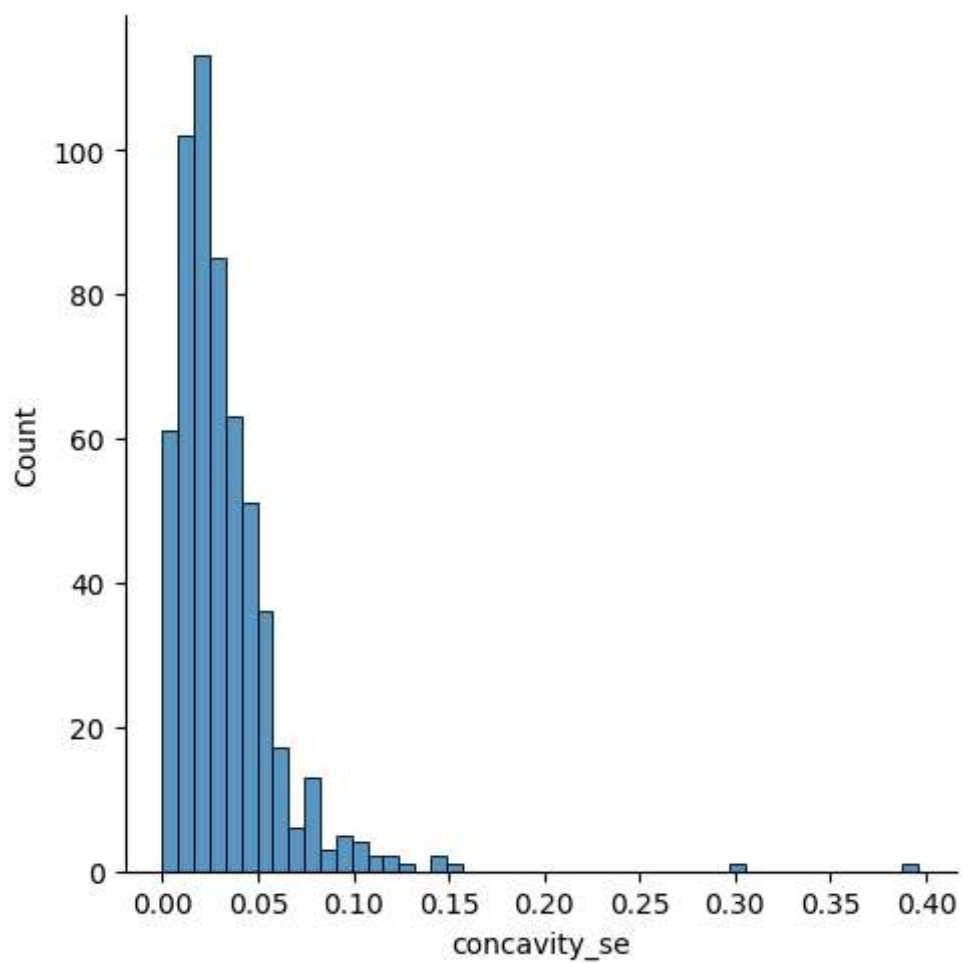


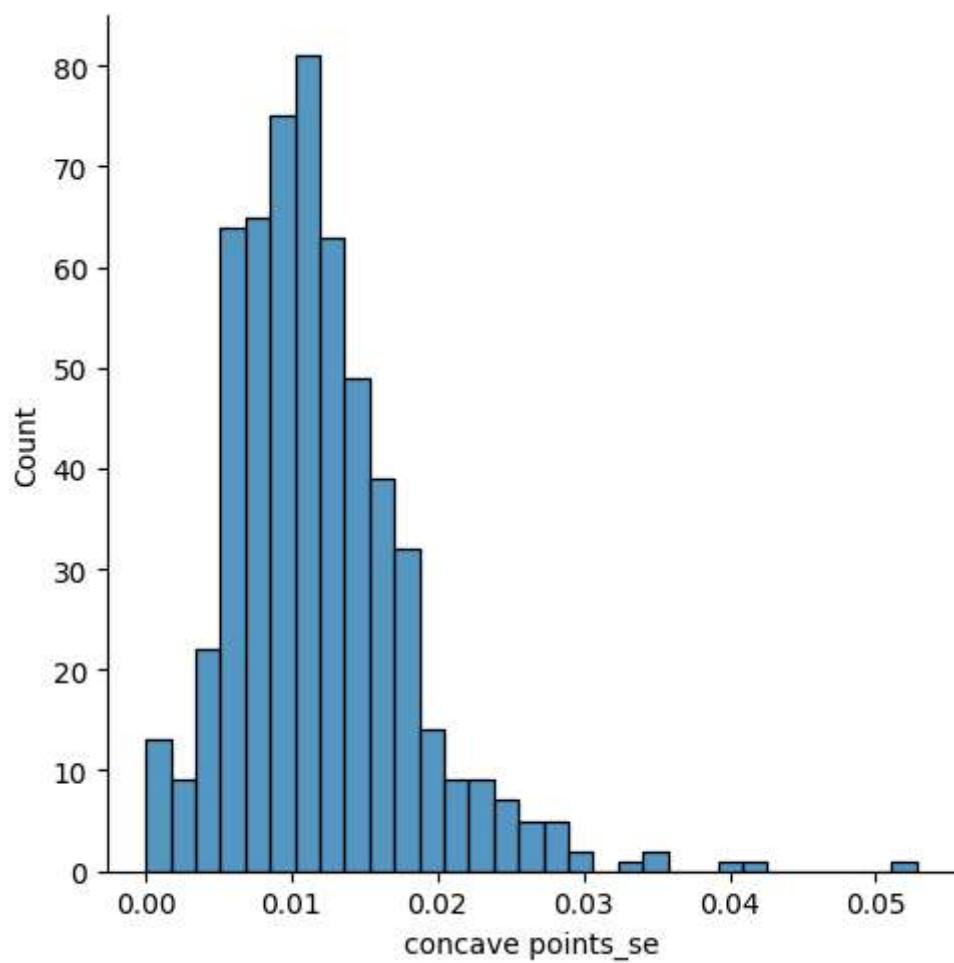


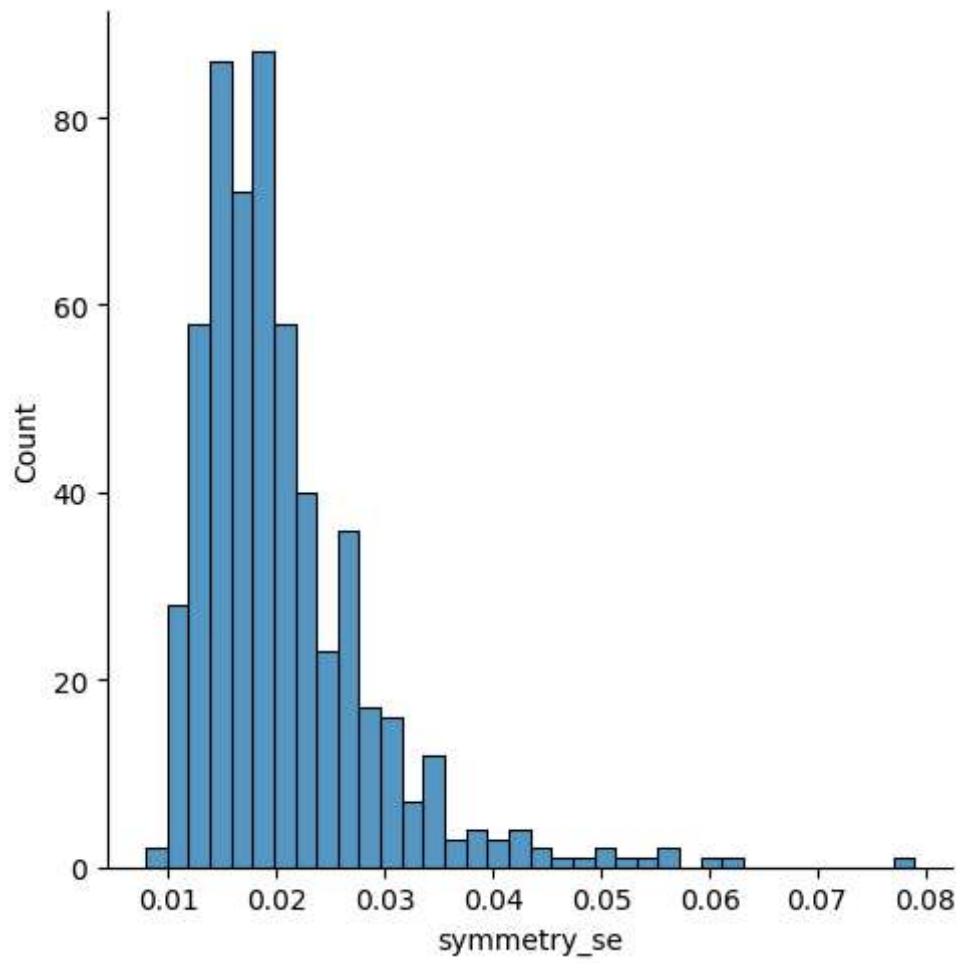


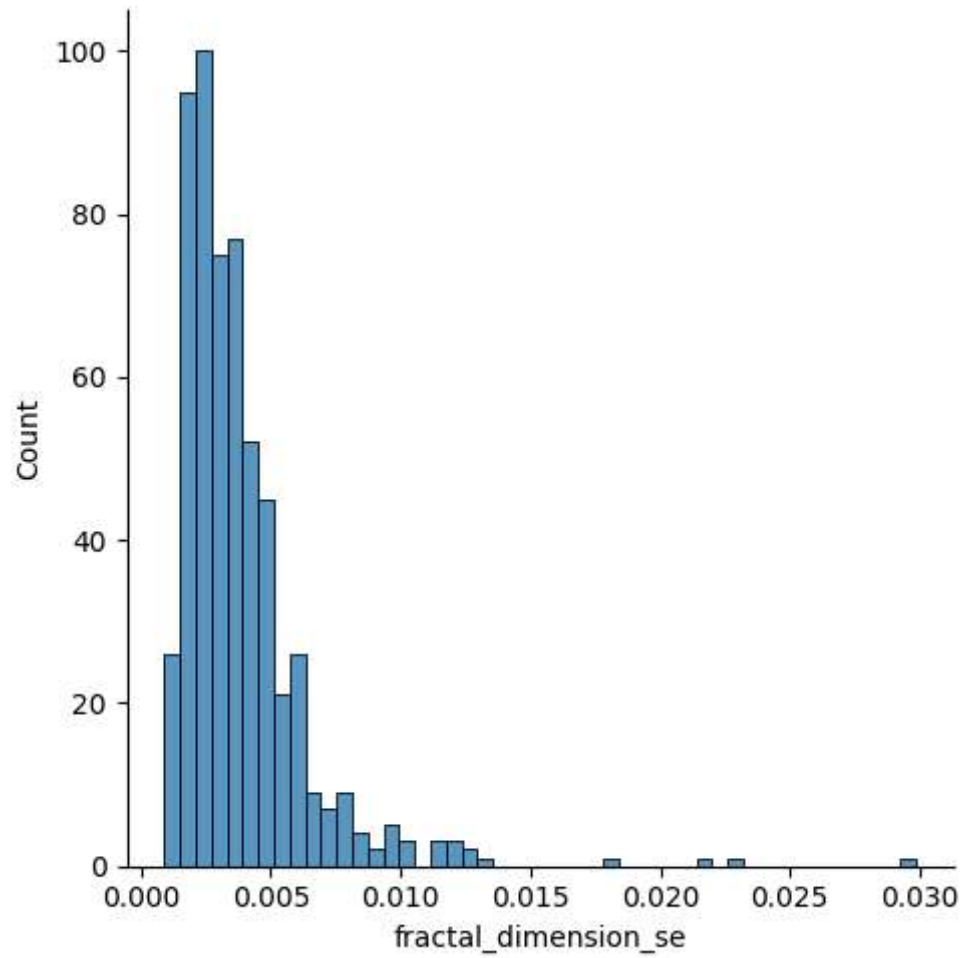


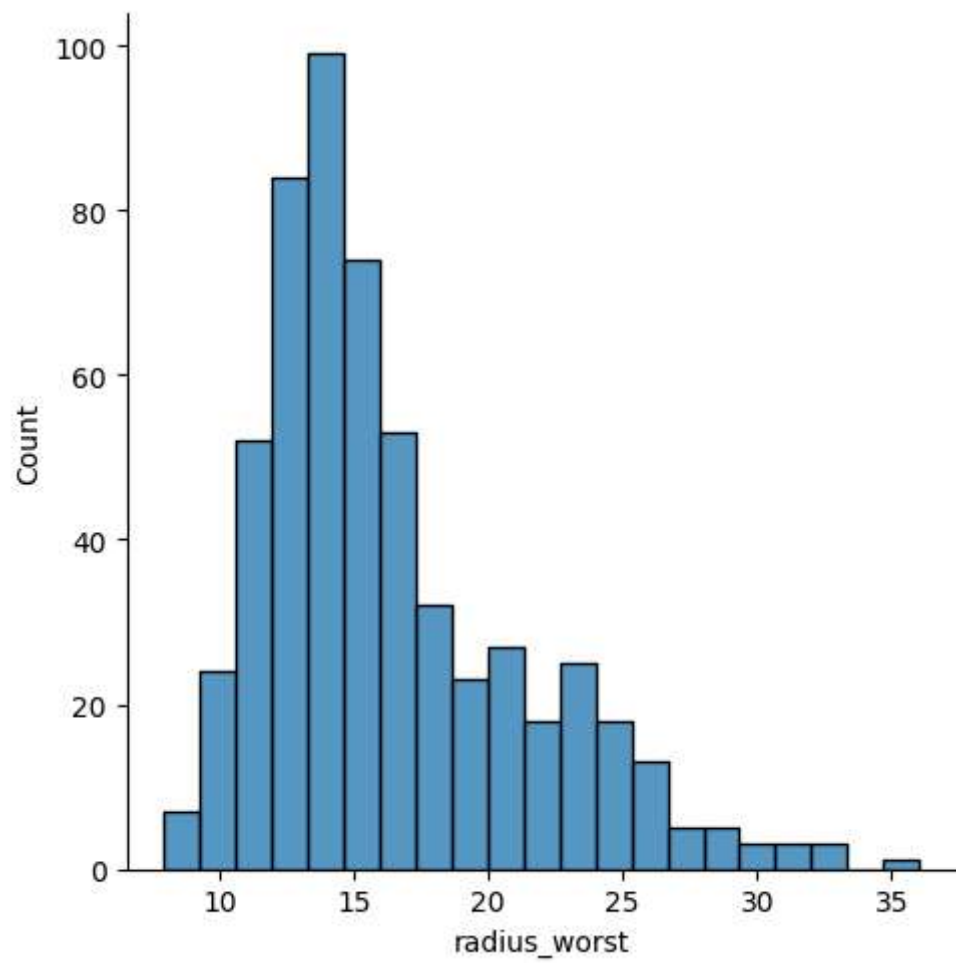


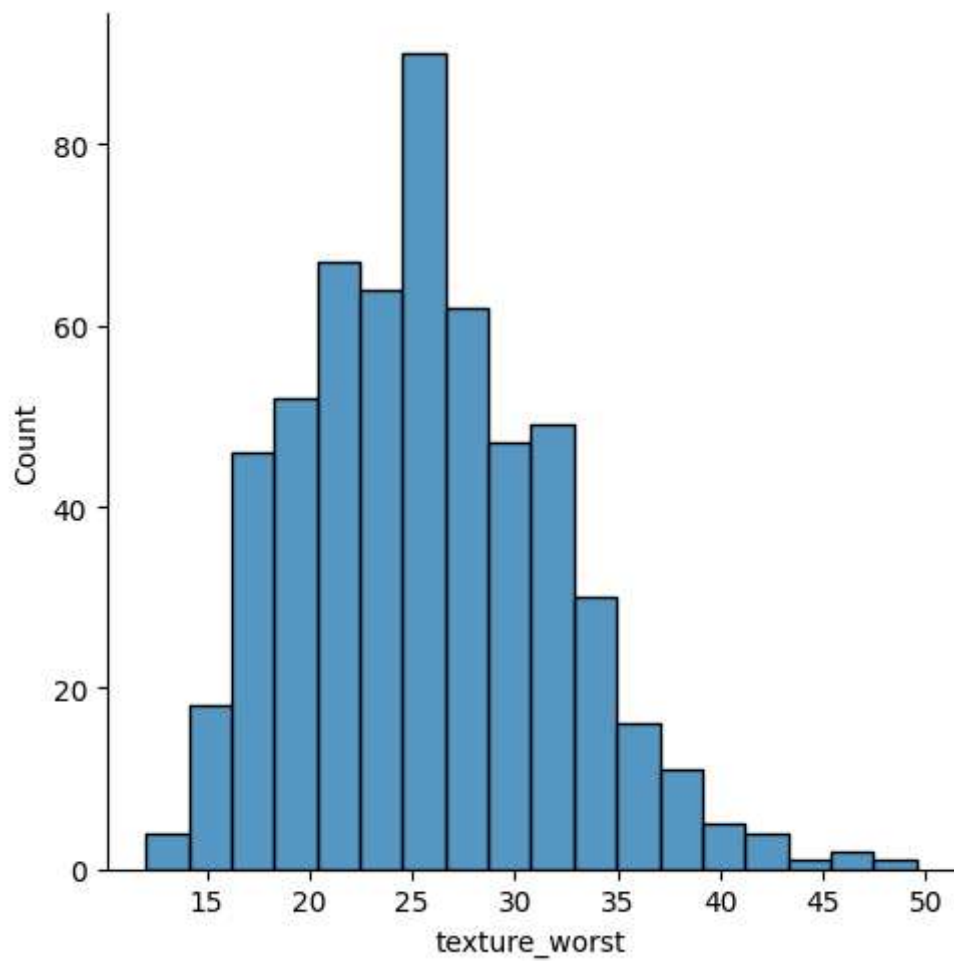


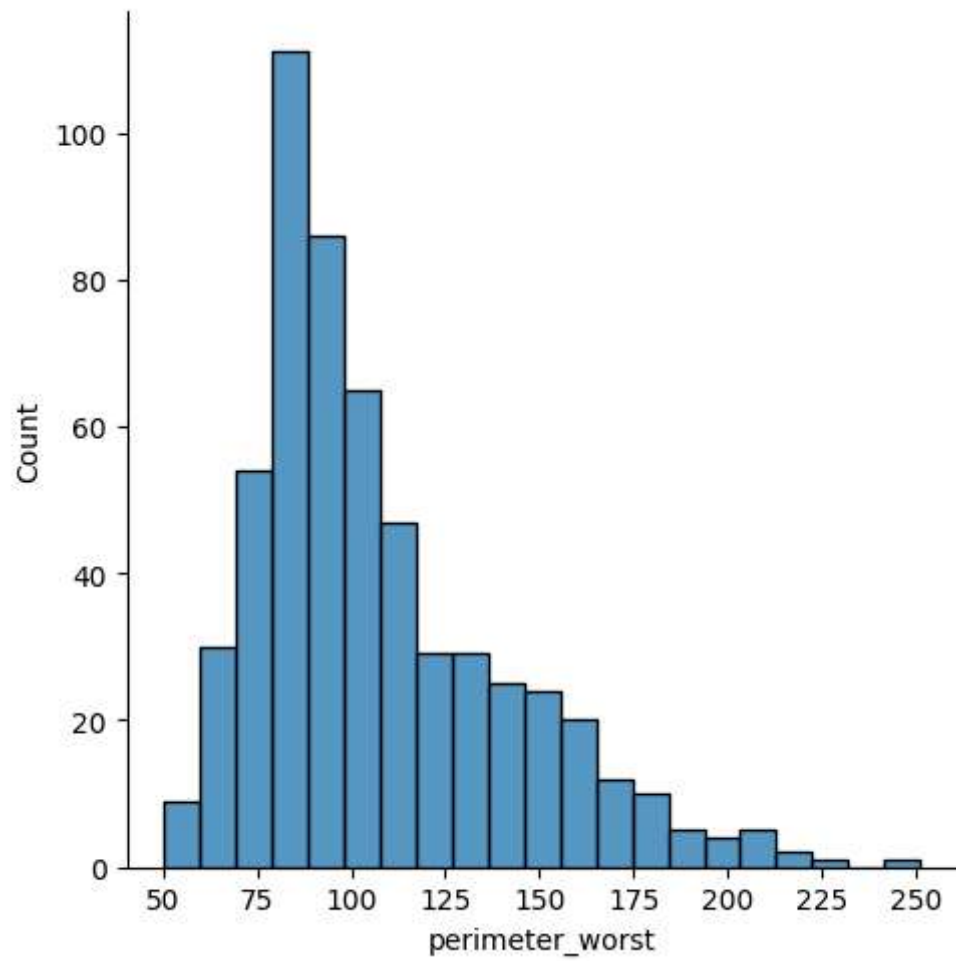


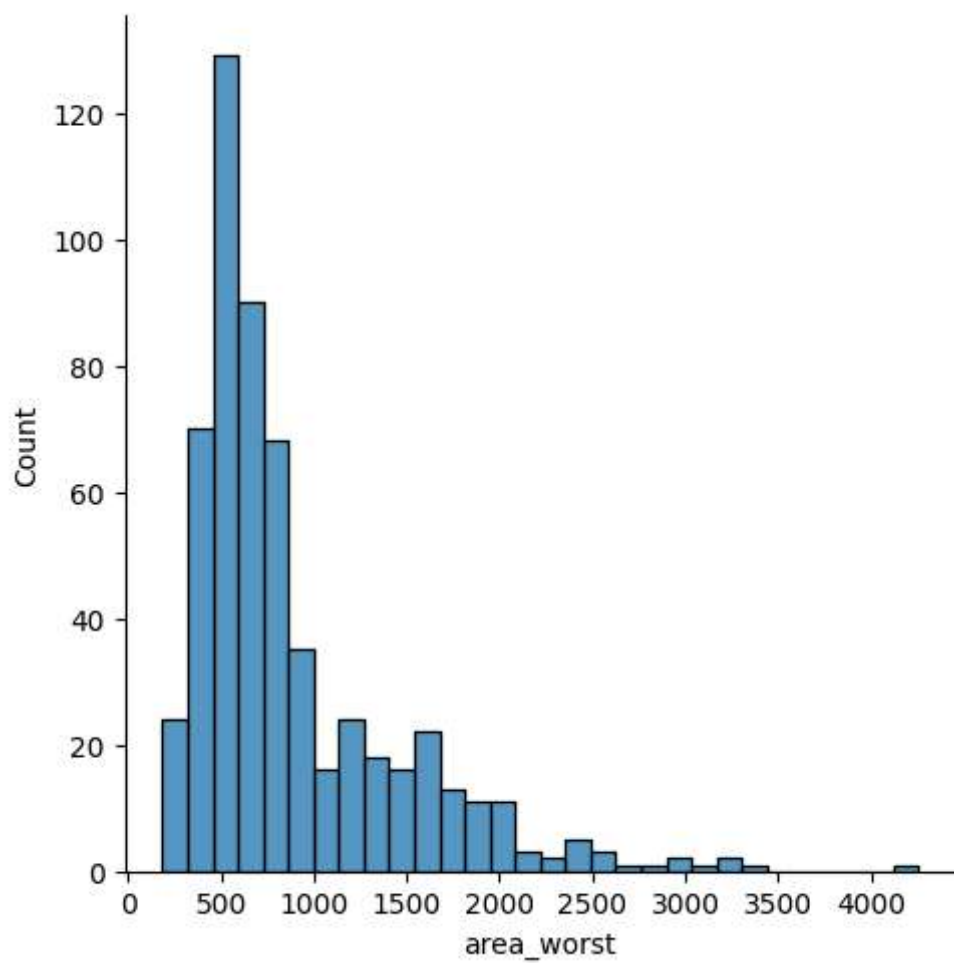


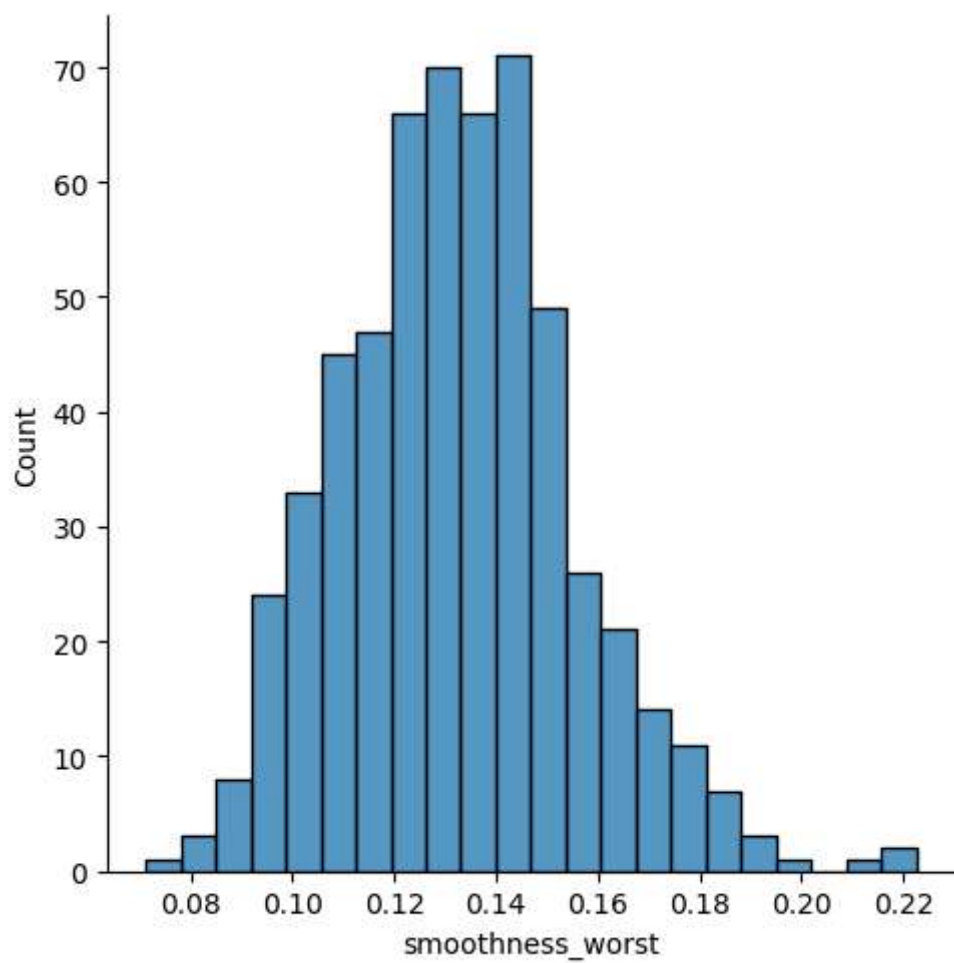


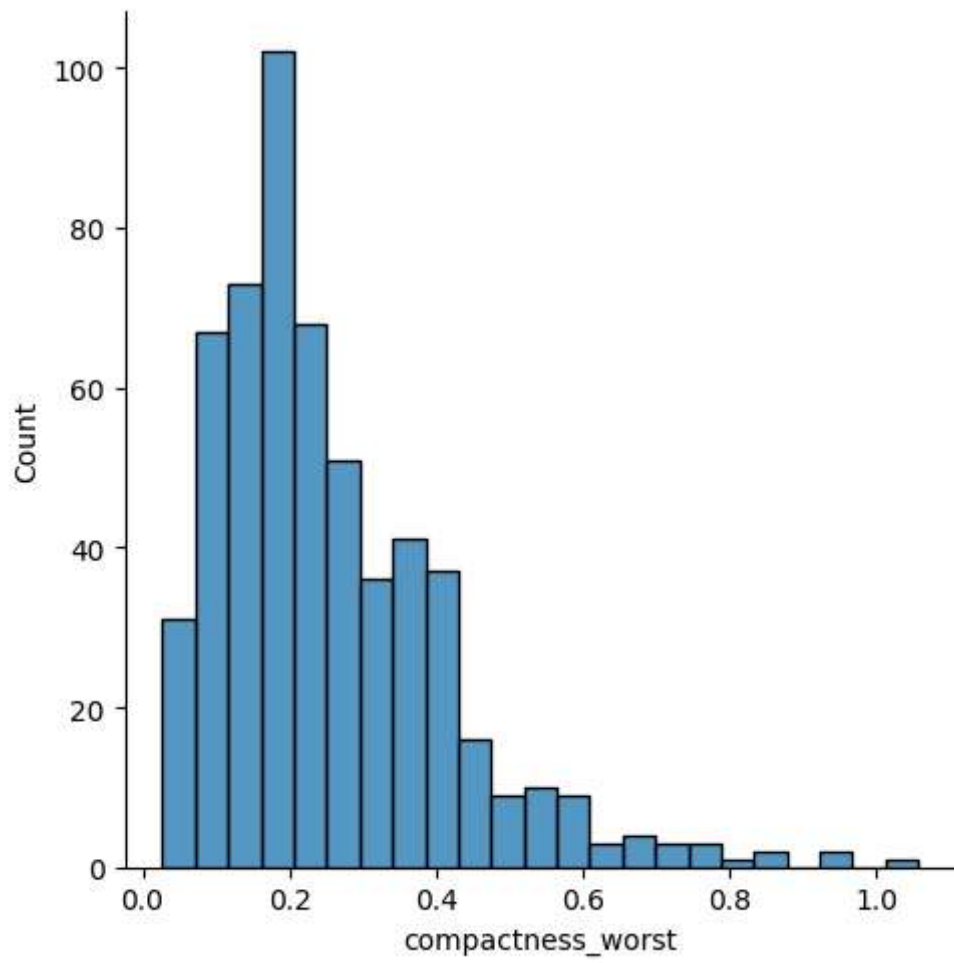


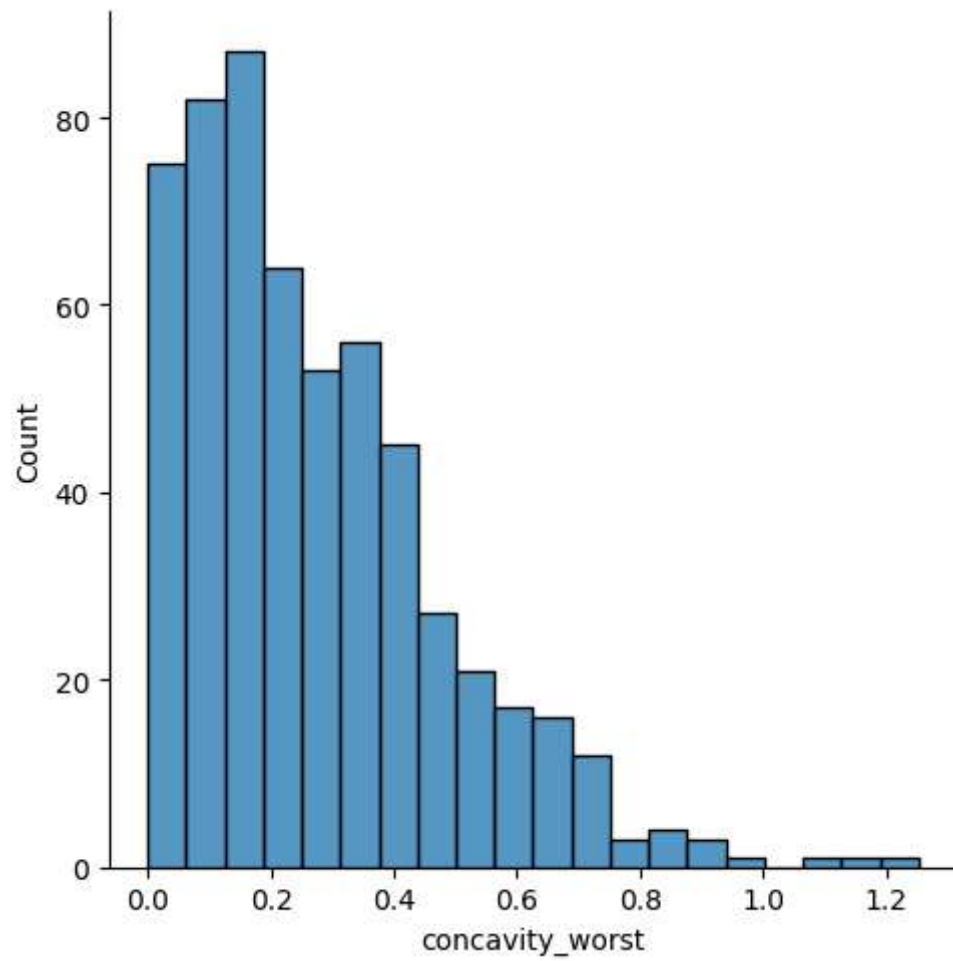


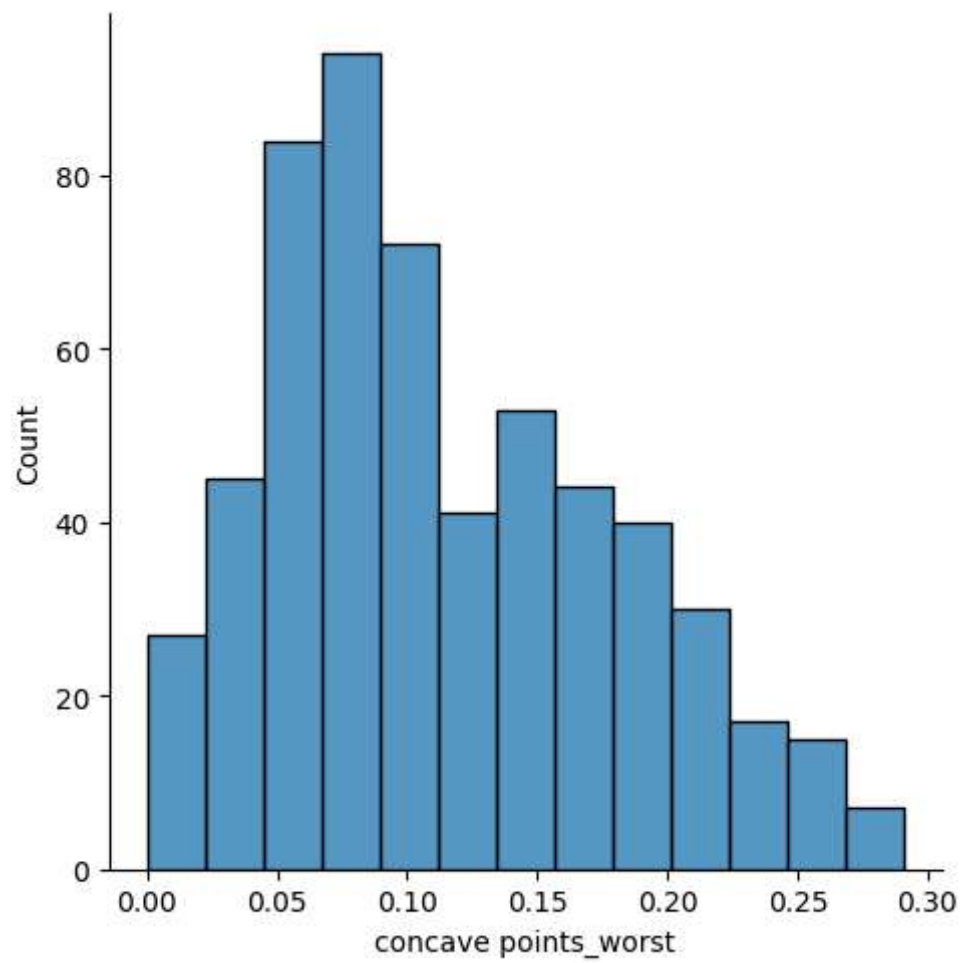


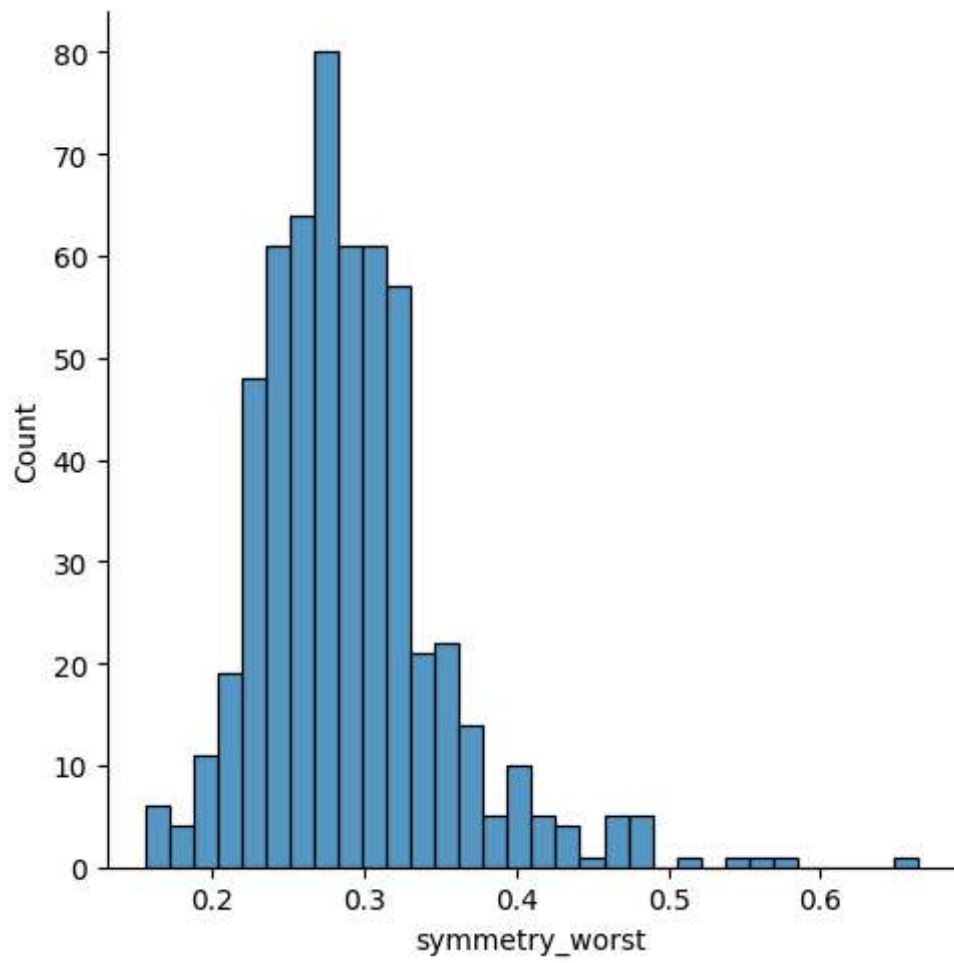


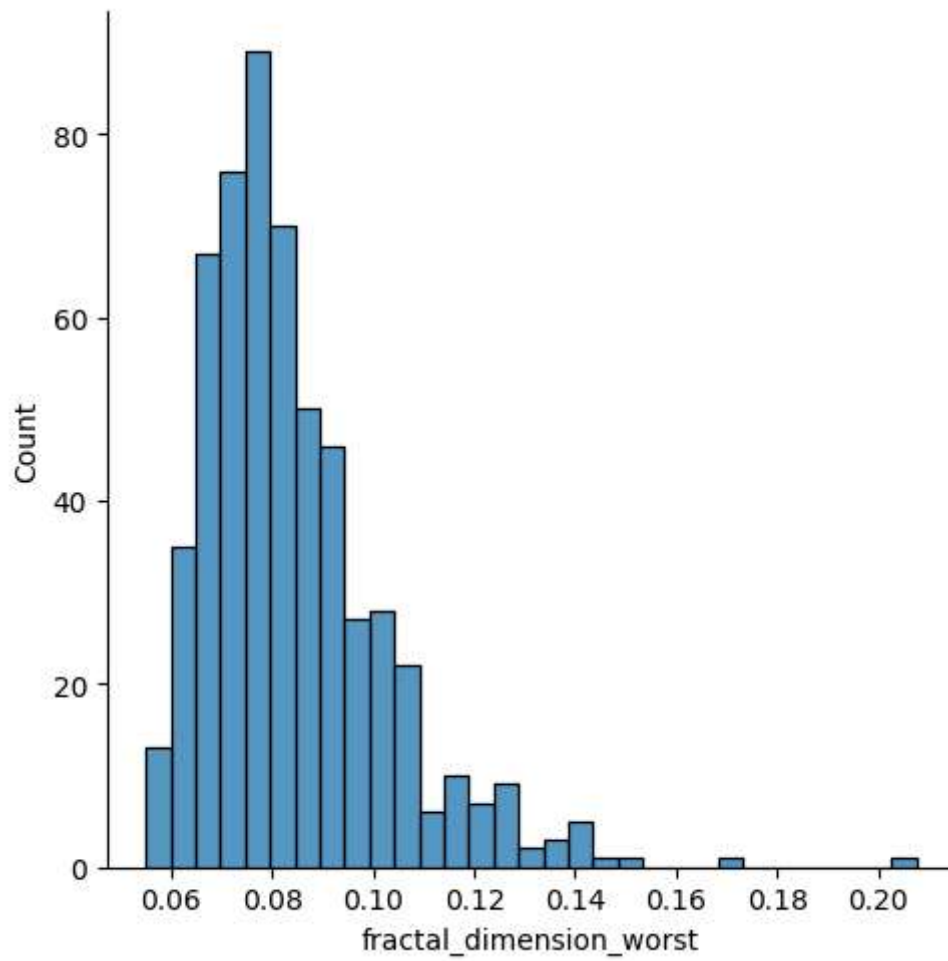


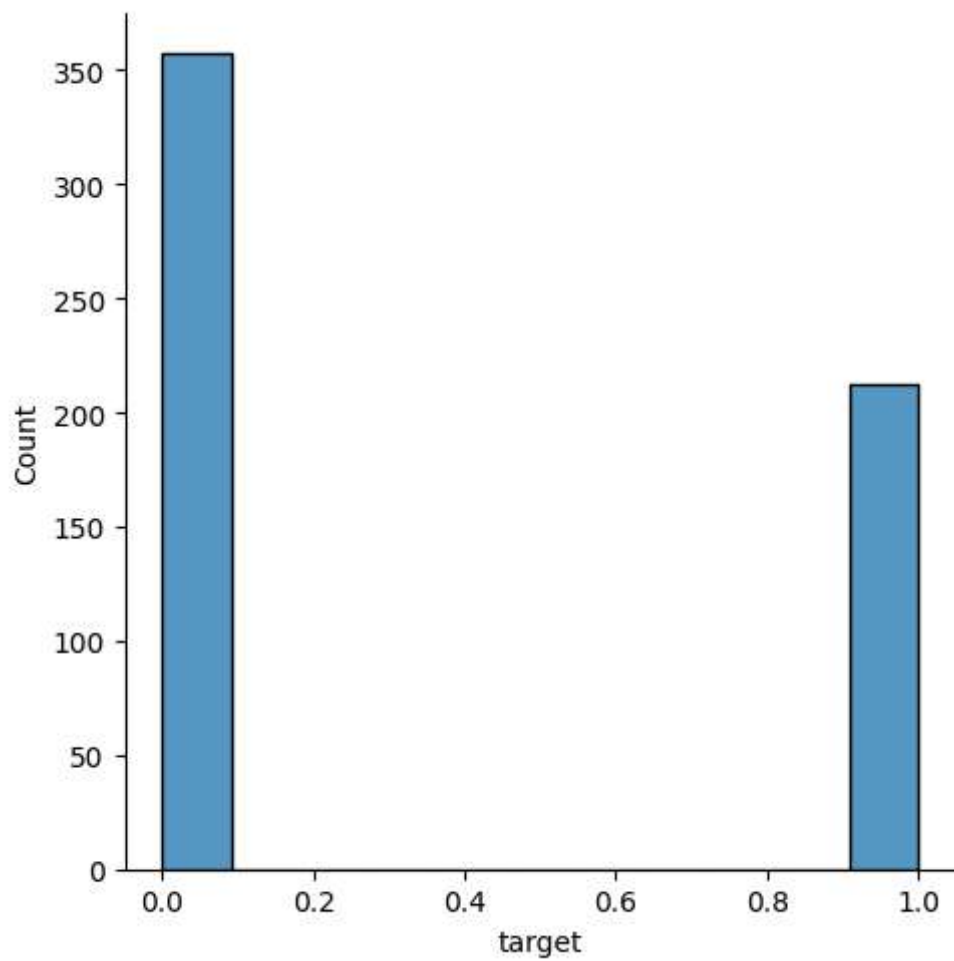






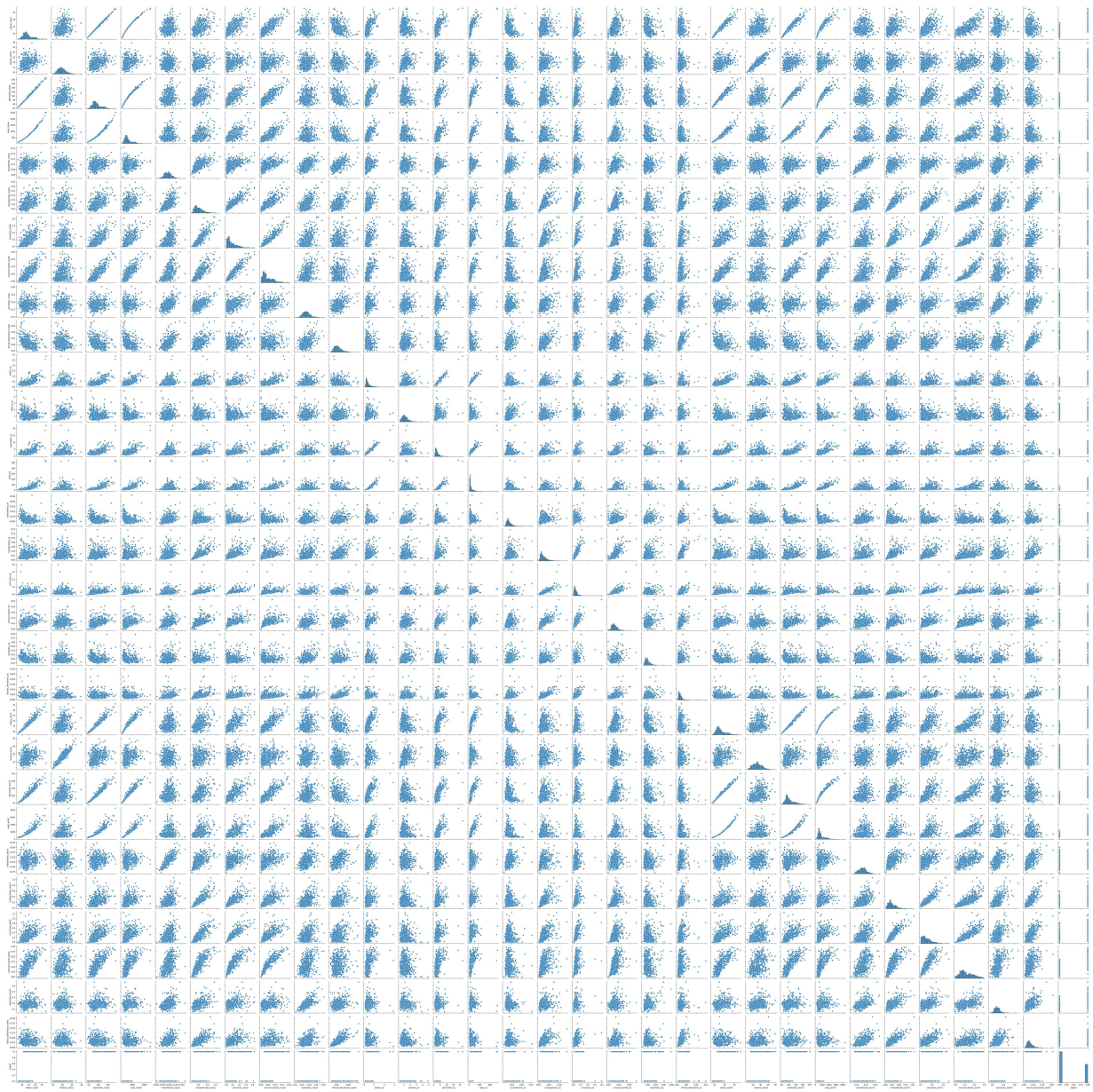






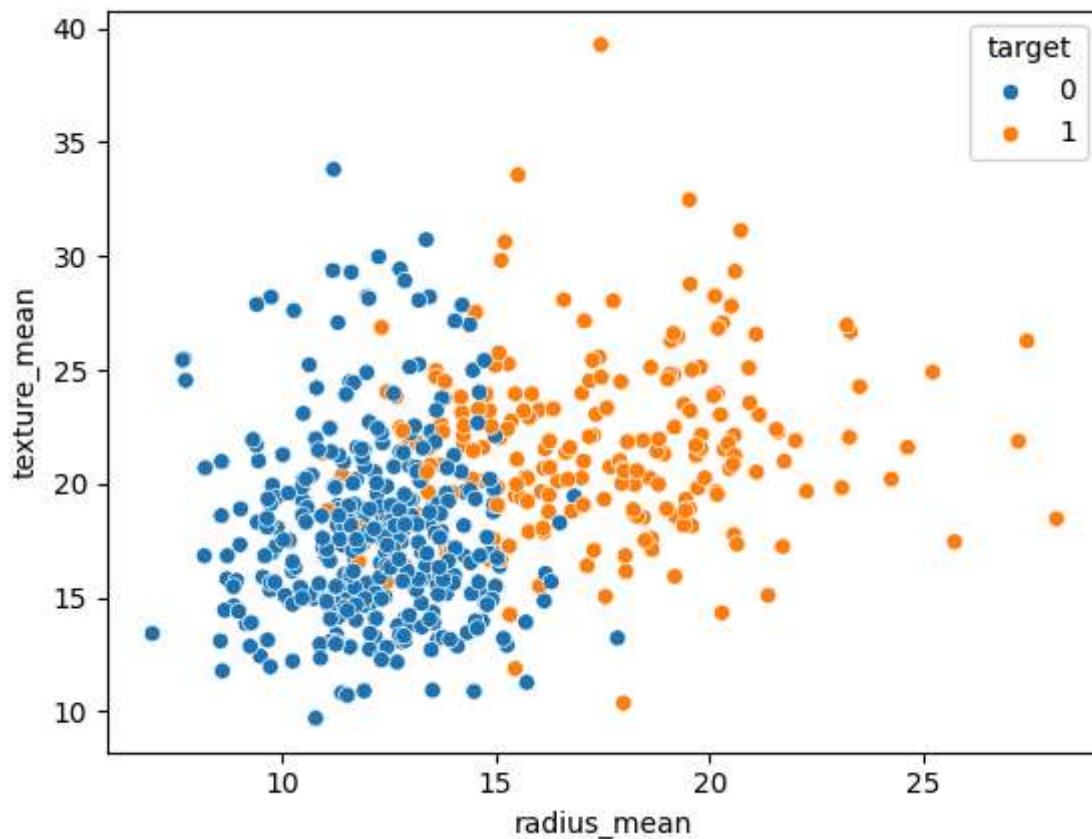
```
In [43]: # pari plot  
sns.pairplot(data)
```

```
Out[43]: <seaborn.axisgrid.PairGrid at 0x1f8a7e6aba0>
```



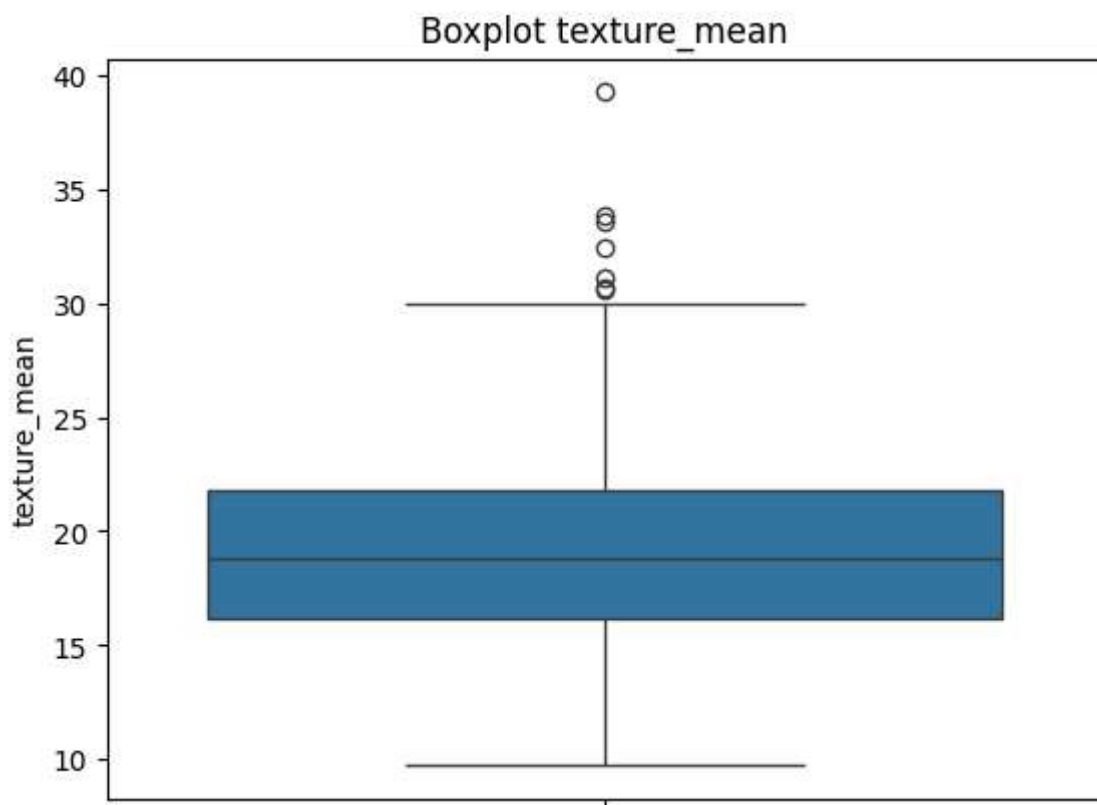
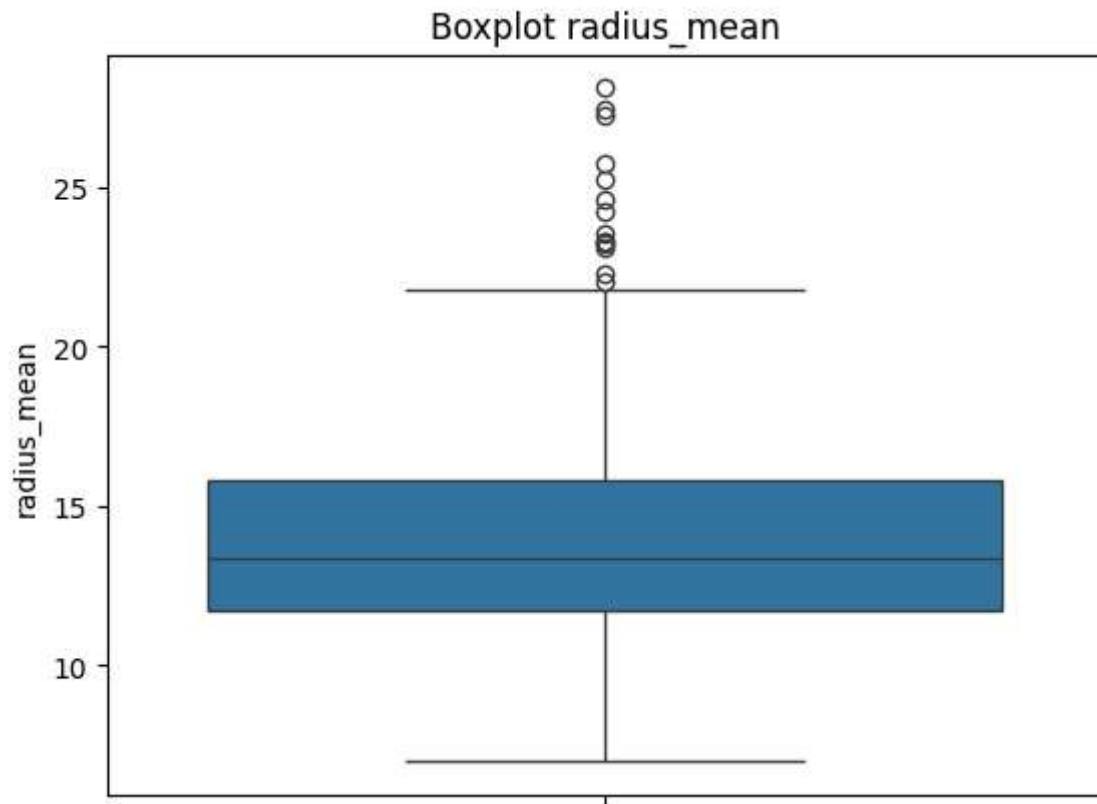
```
In [45]: # scatterplot for first 2 columns
first_column = data.iloc[:,0]
second_column = data.iloc[:,1]
sns.scatterplot(x=first_column, y=second_column, data=data, hue='target')
```

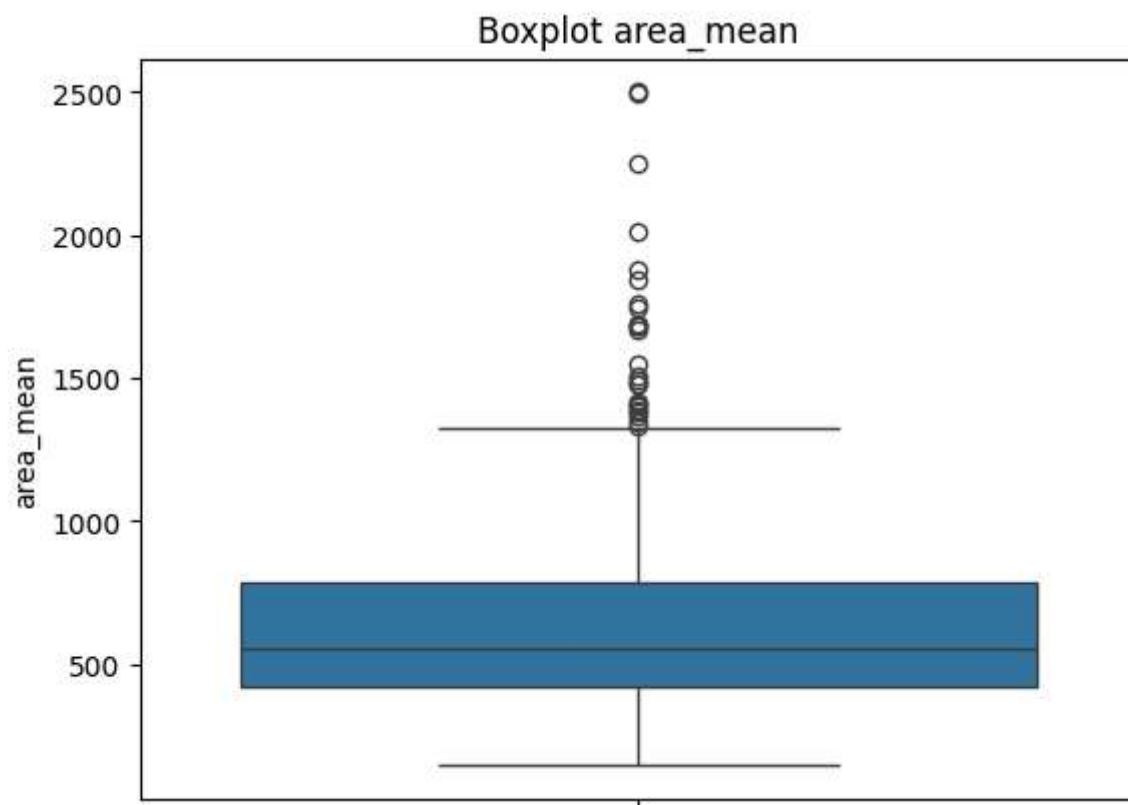
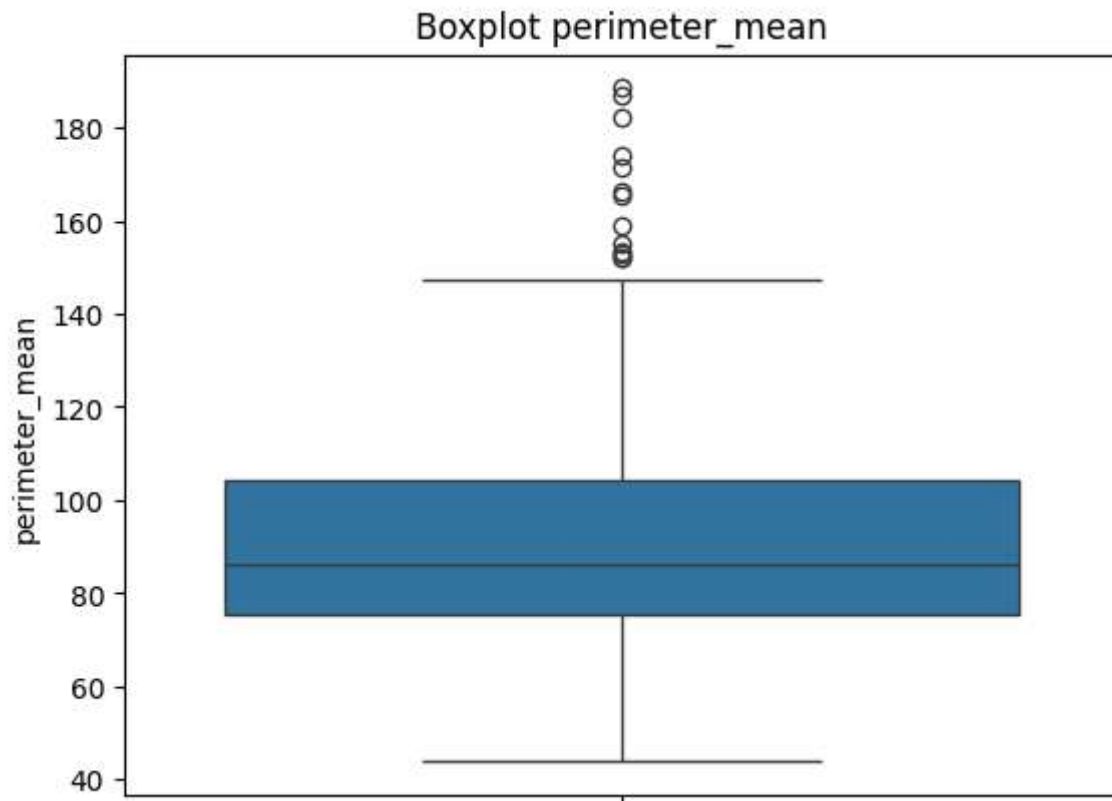
```
Out[45]: <Axes: xlabel='radius_mean', ylabel='texture_mean'>
```

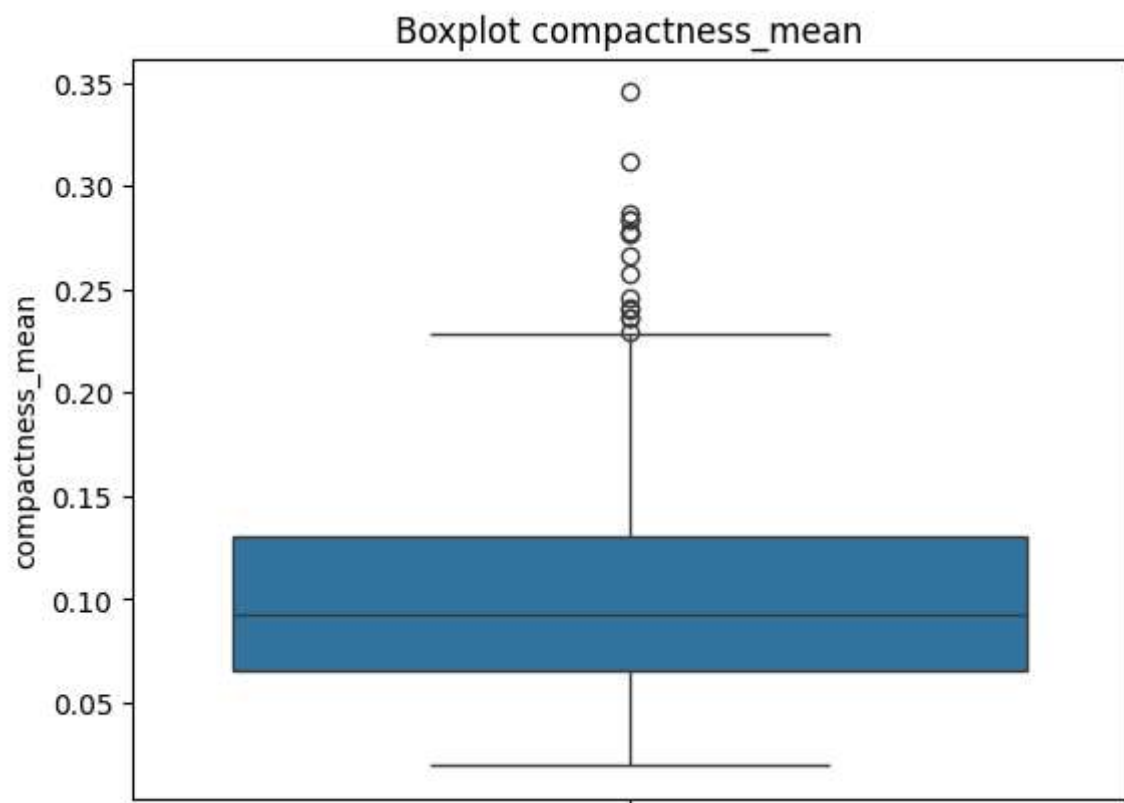
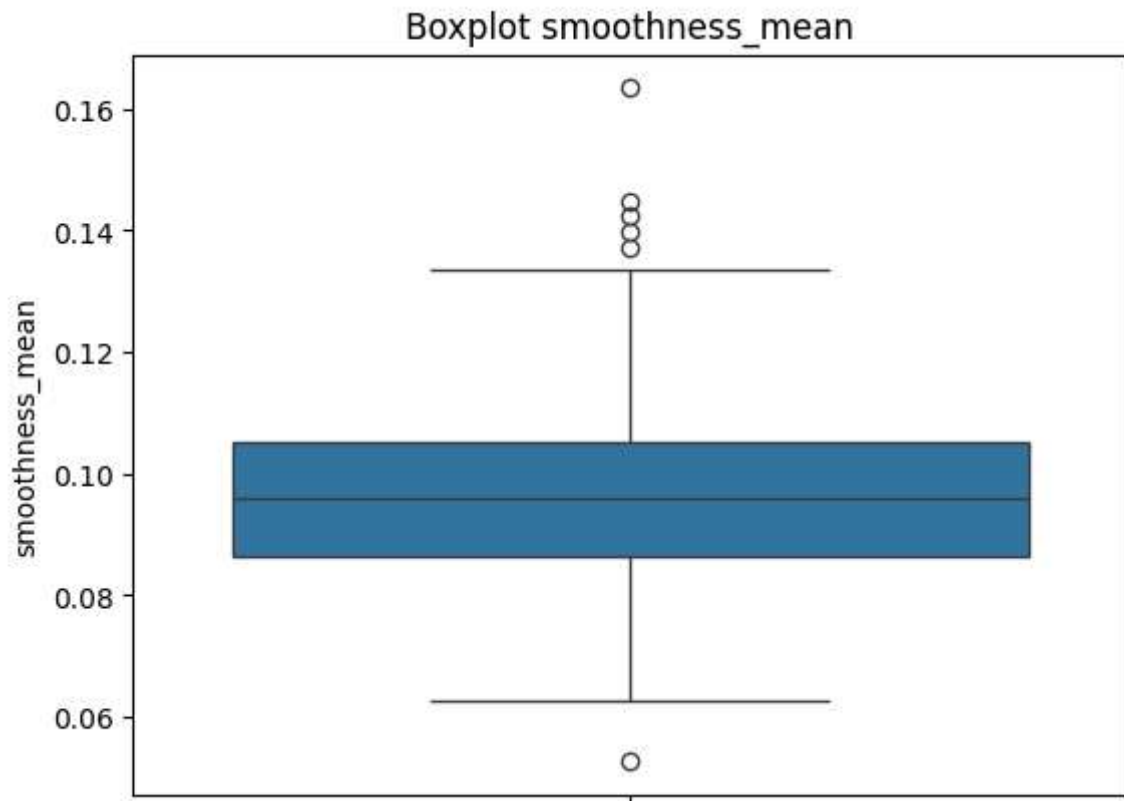


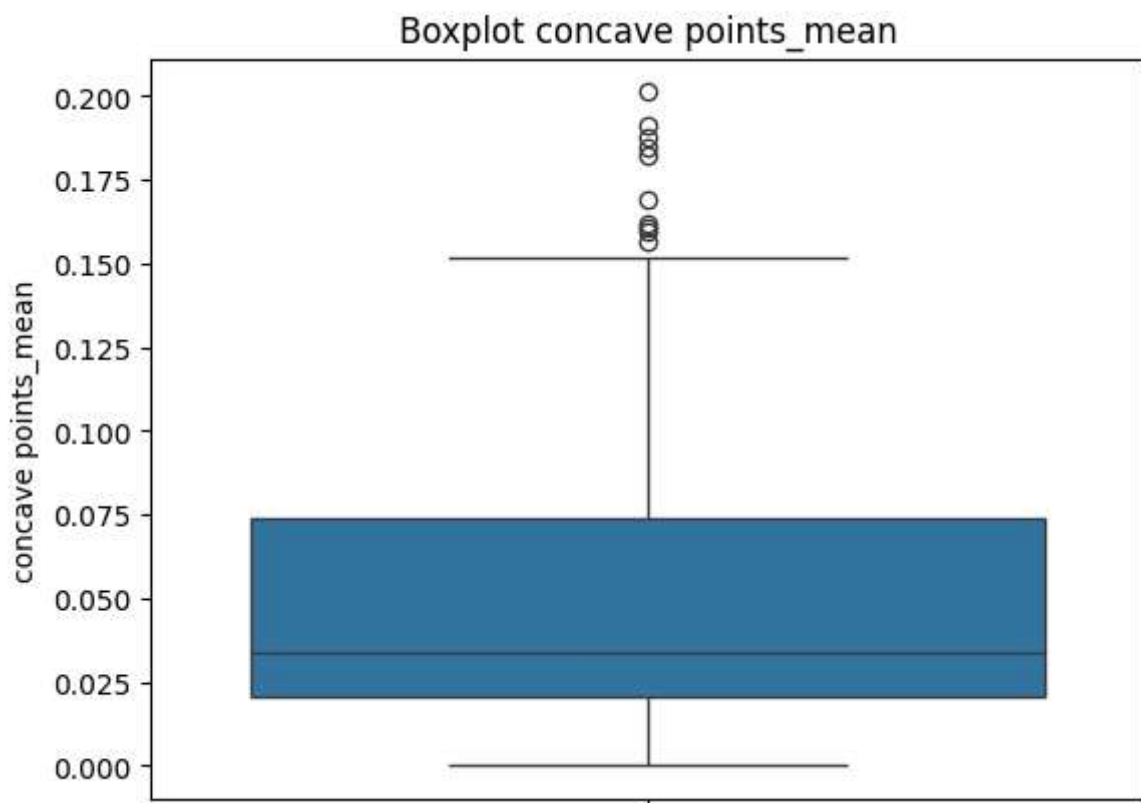
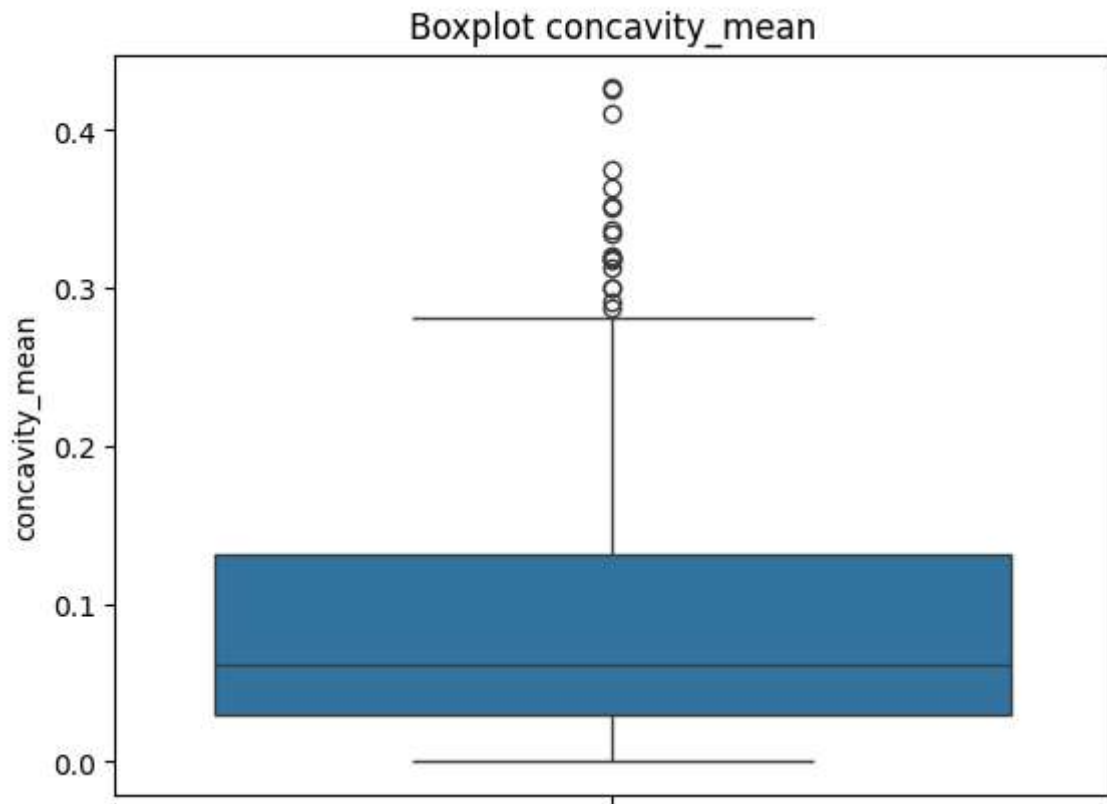
Outliers Detection

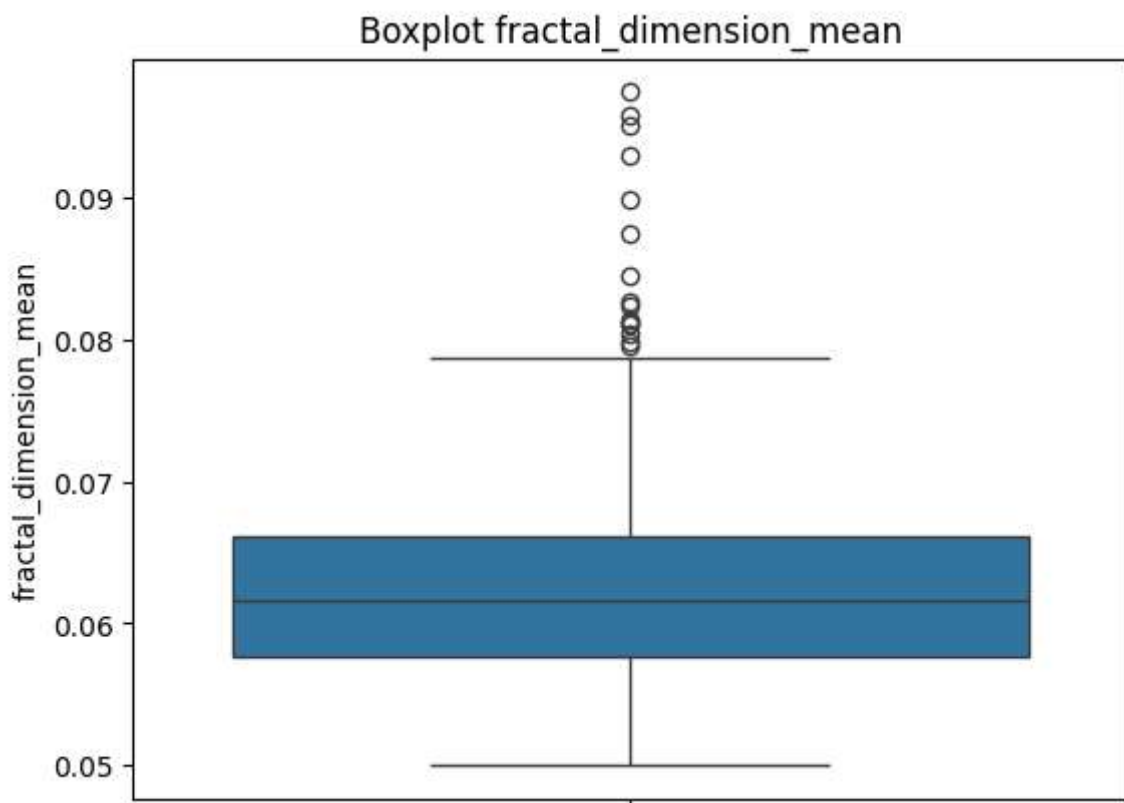
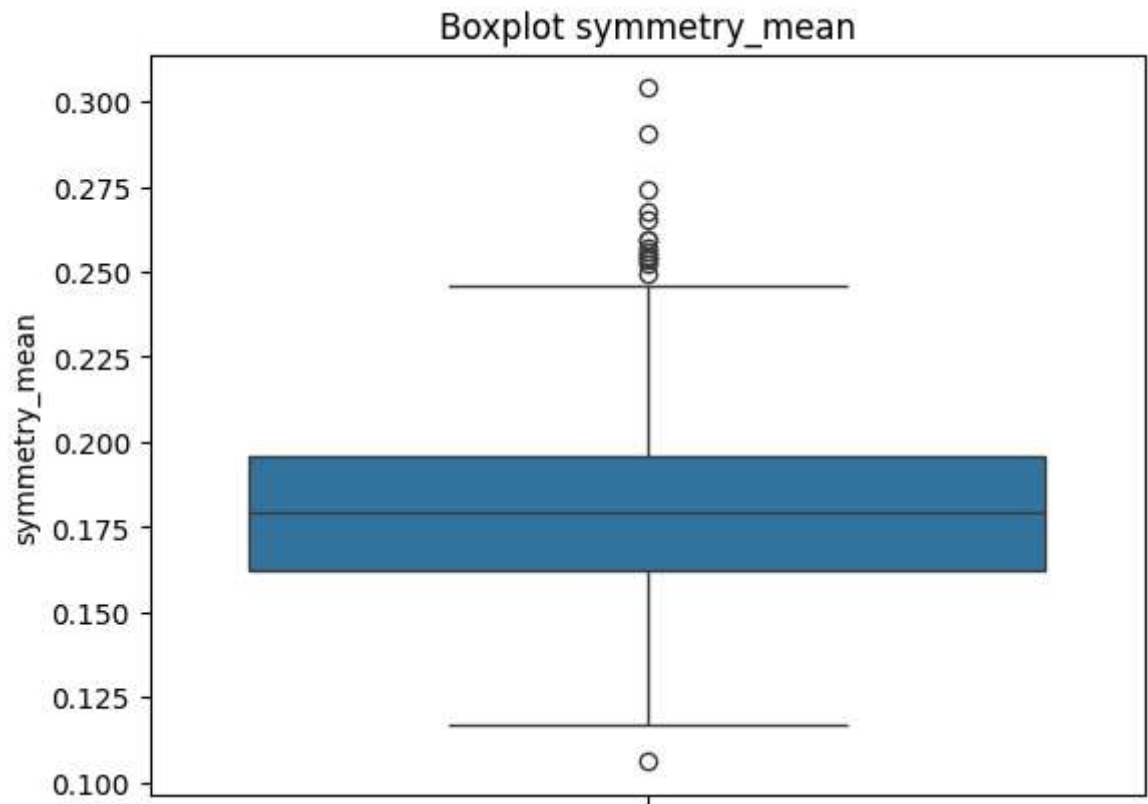
```
In [47]: for col in data:
          sns.boxplot(data=data, y=col)
          plt.title(f"Boxplot {col}")
          plt.ylabel(col)
          plt.show()
```

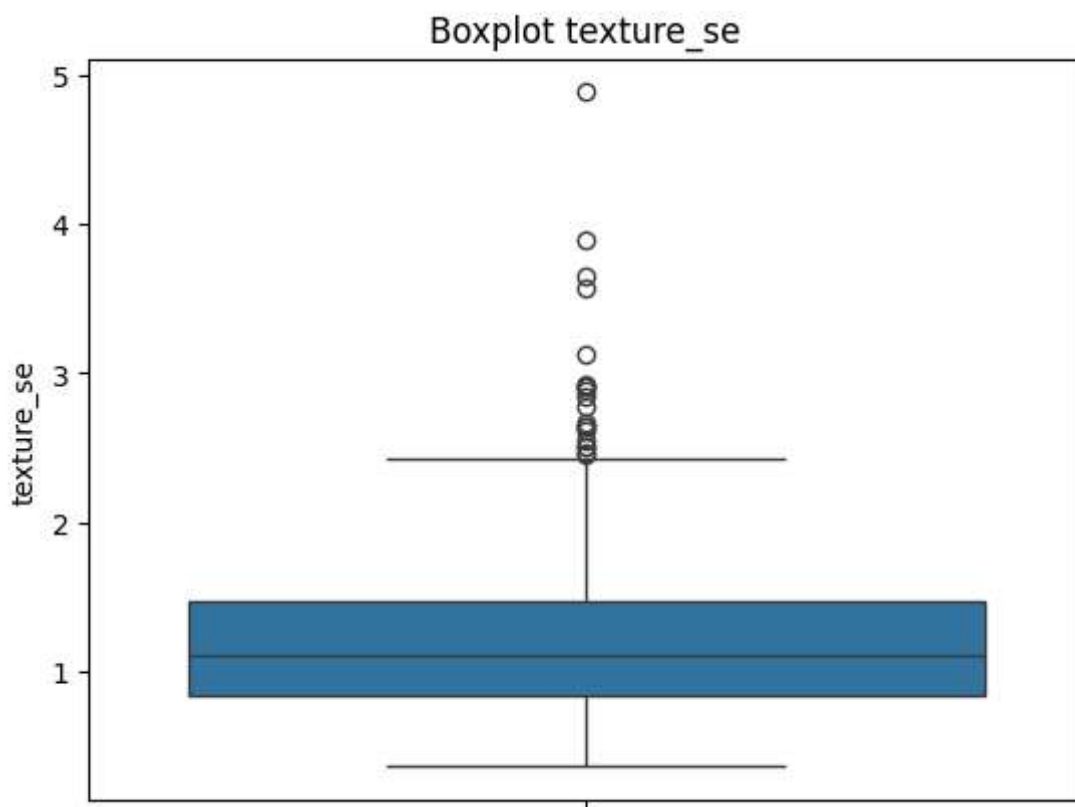
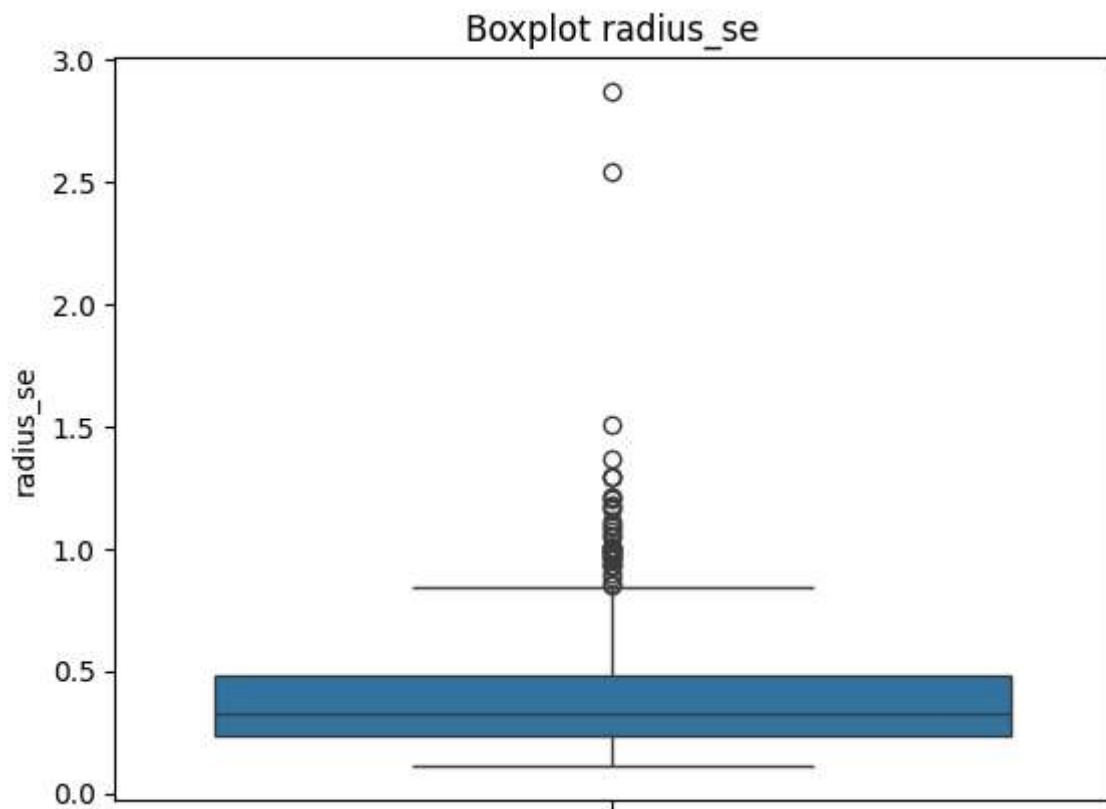


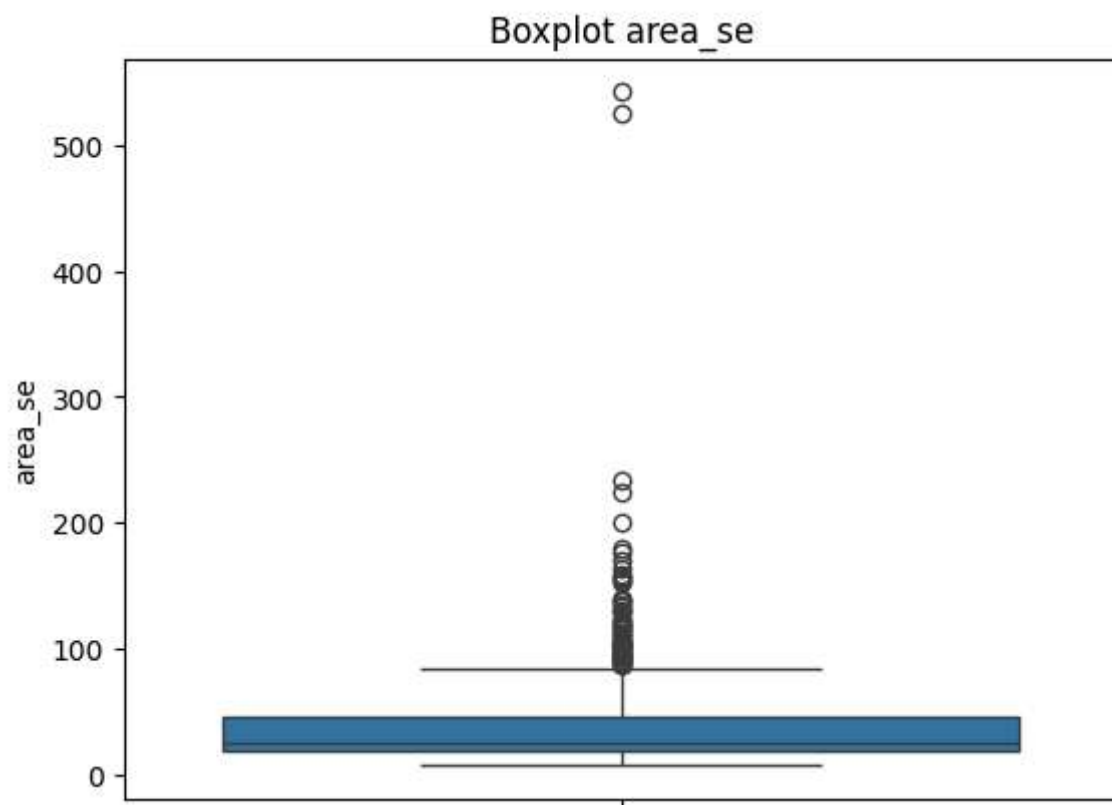
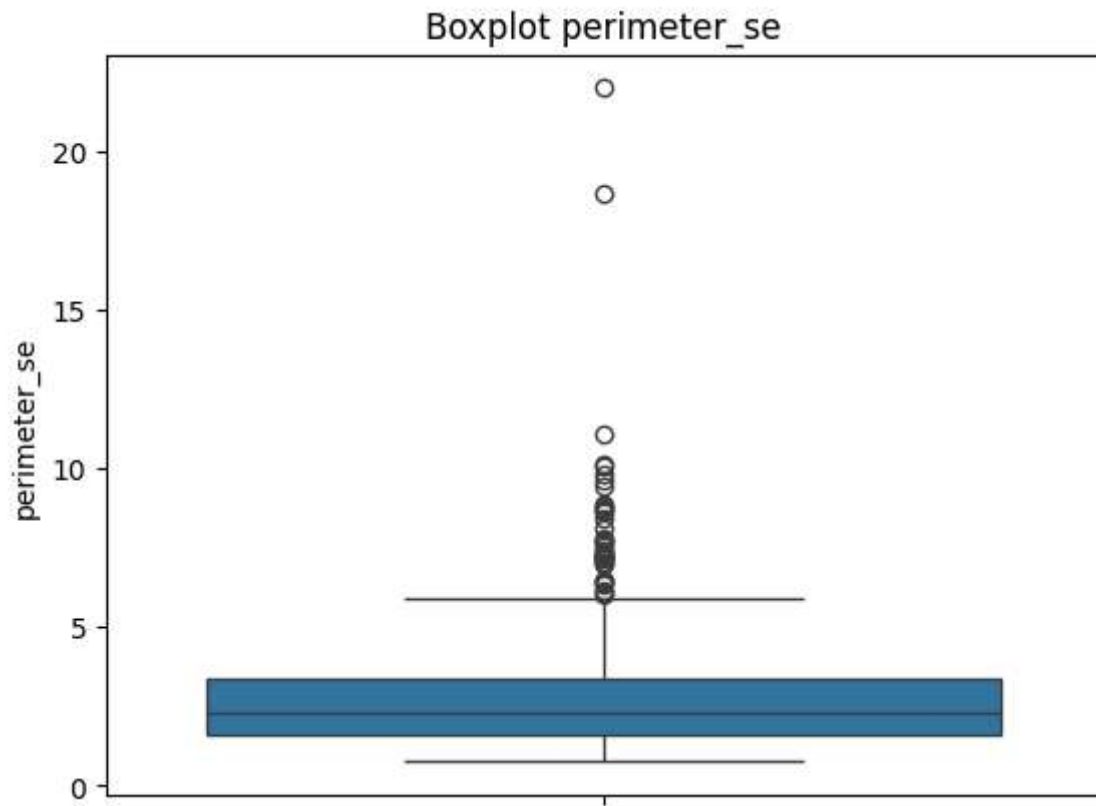


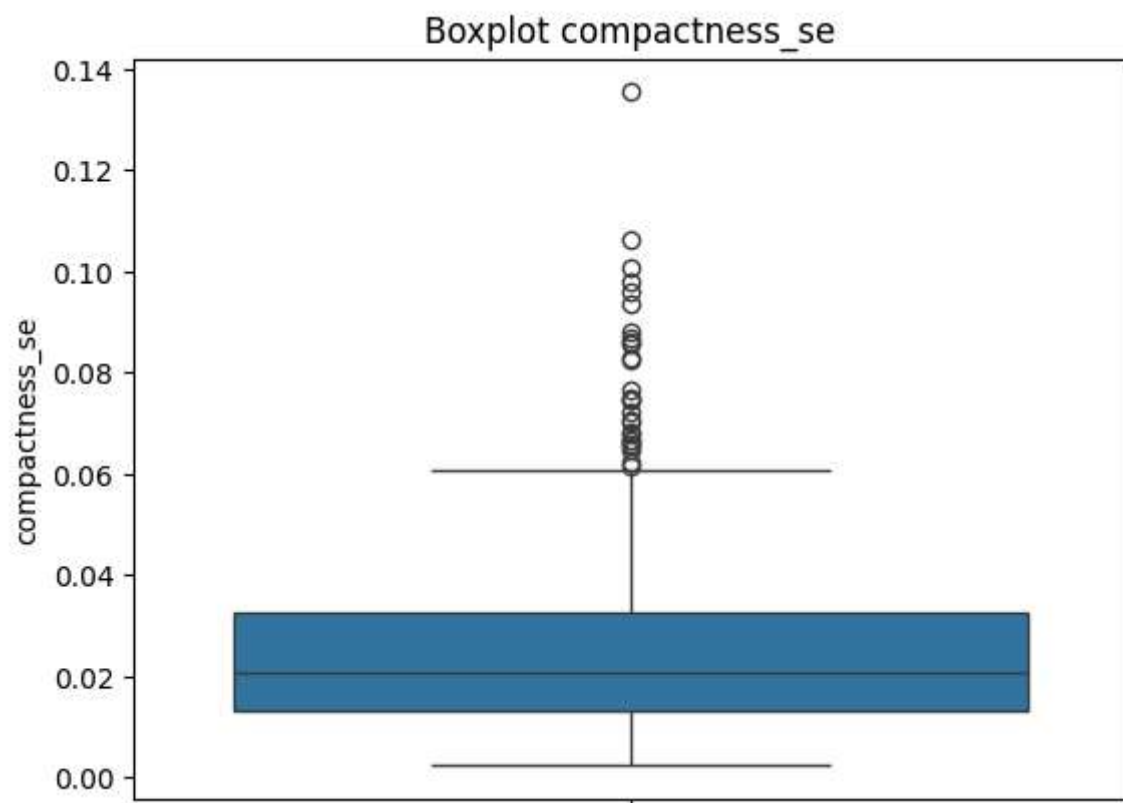
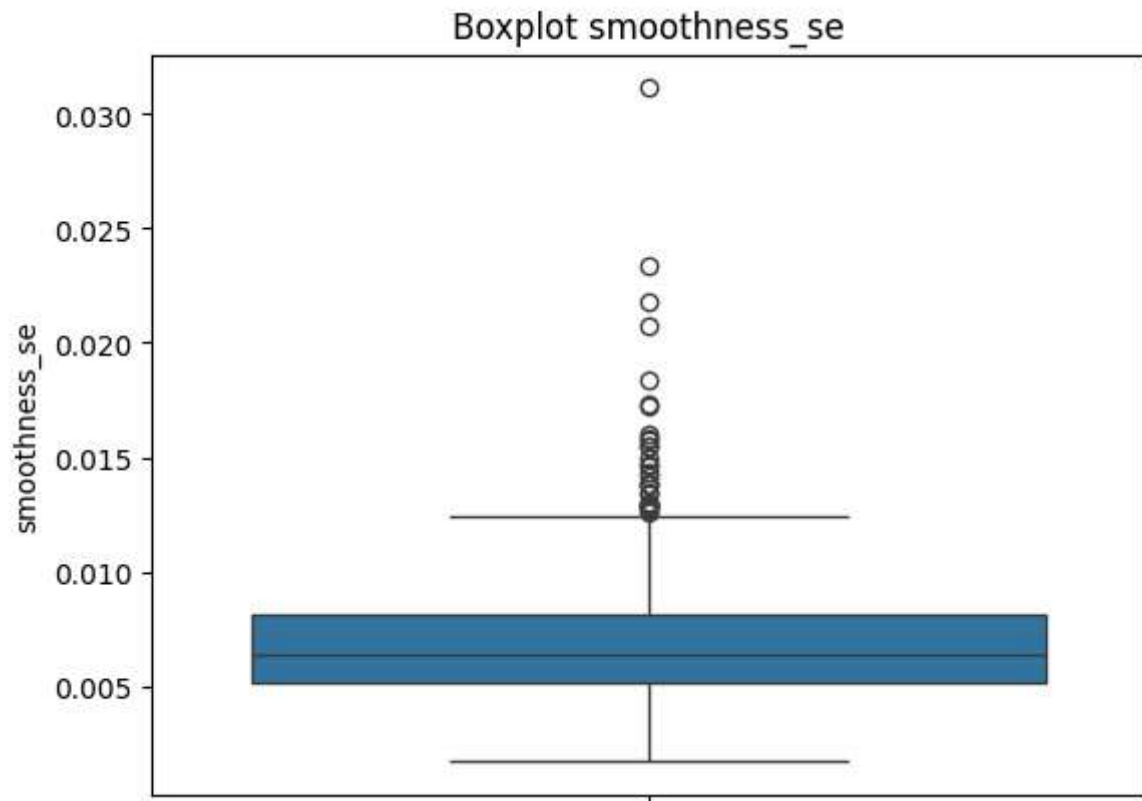


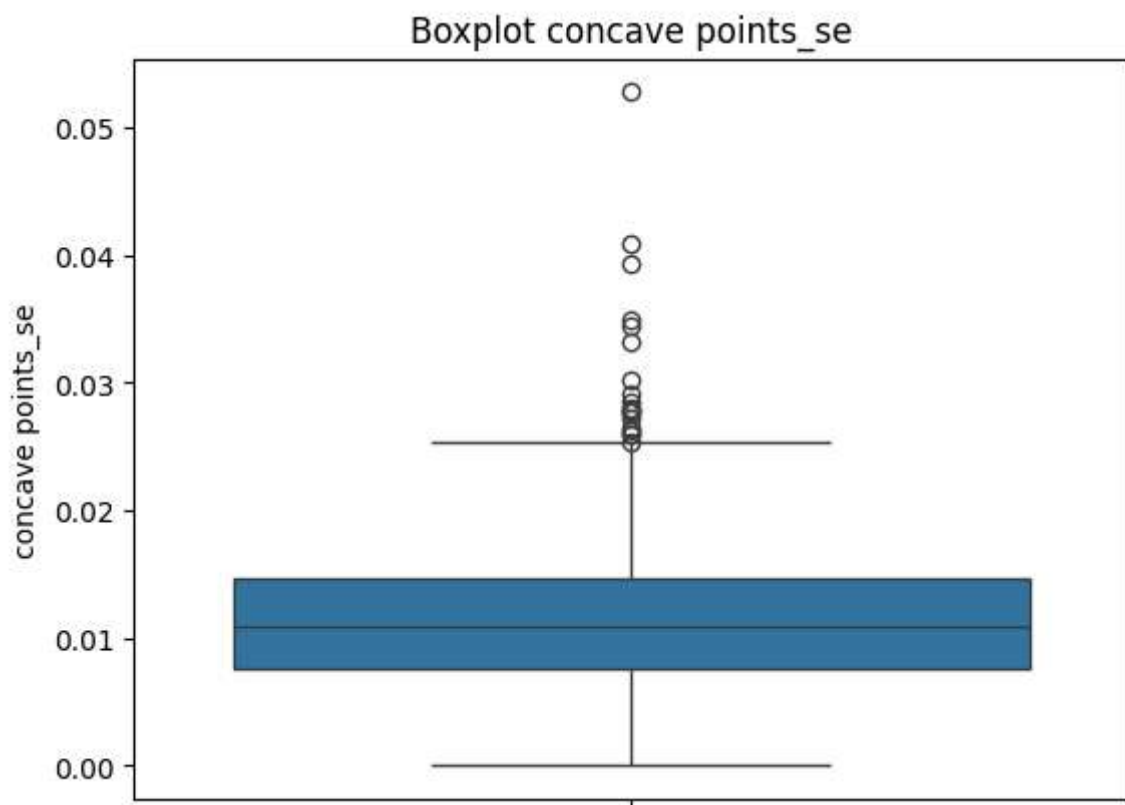
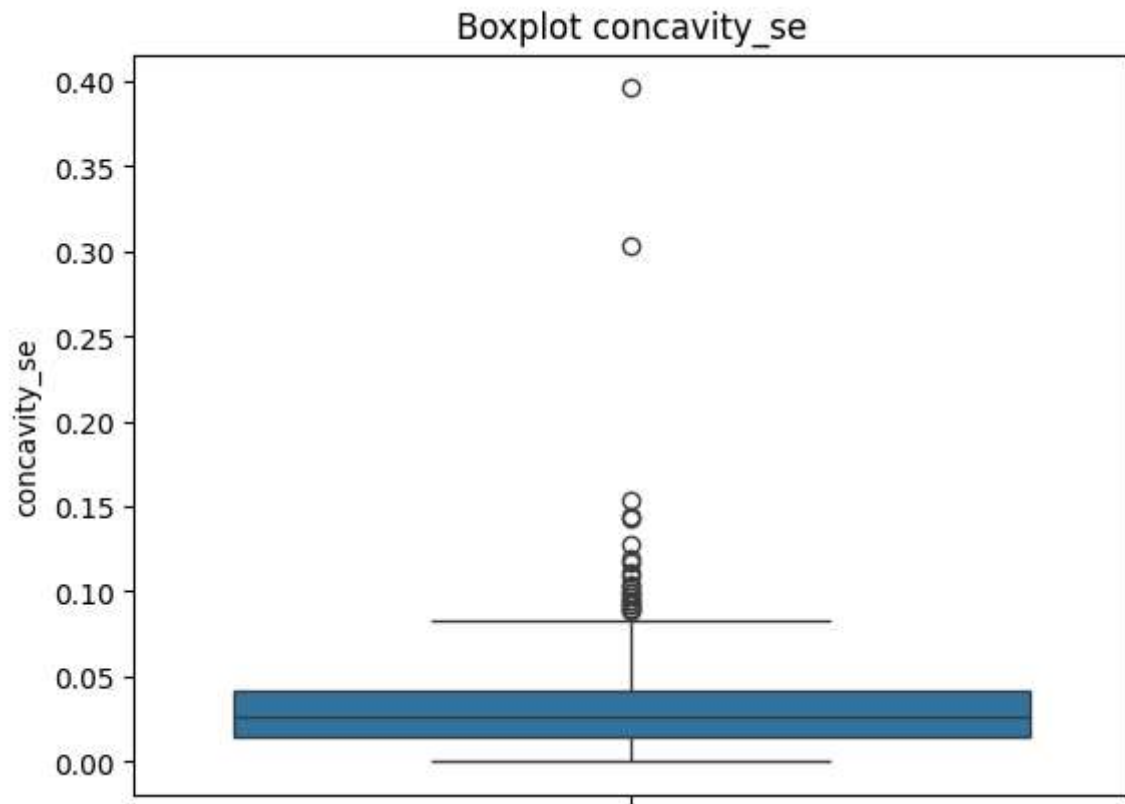


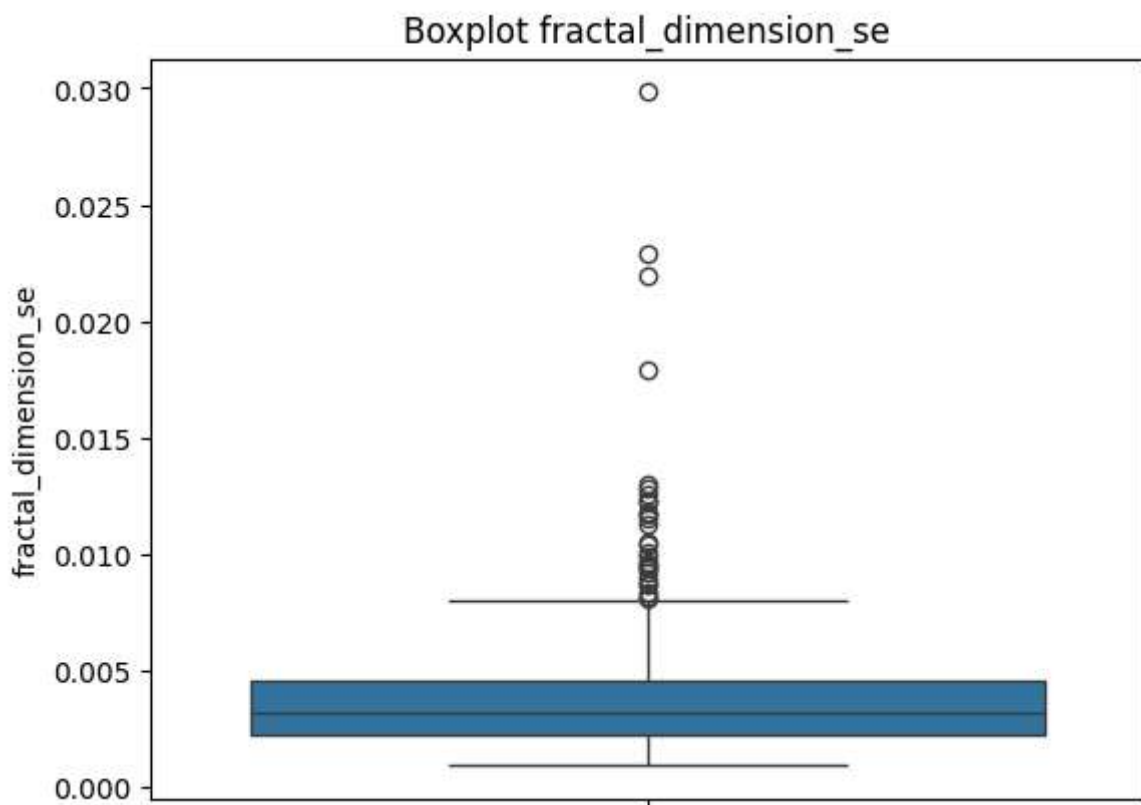
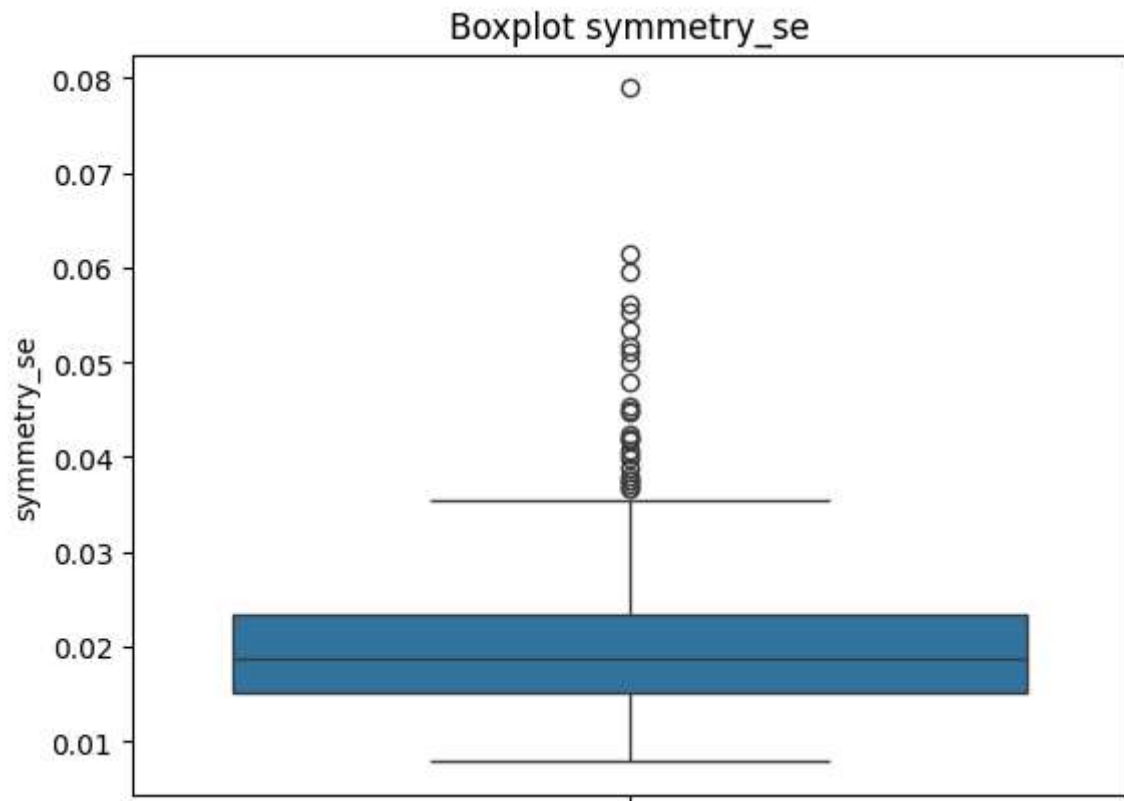


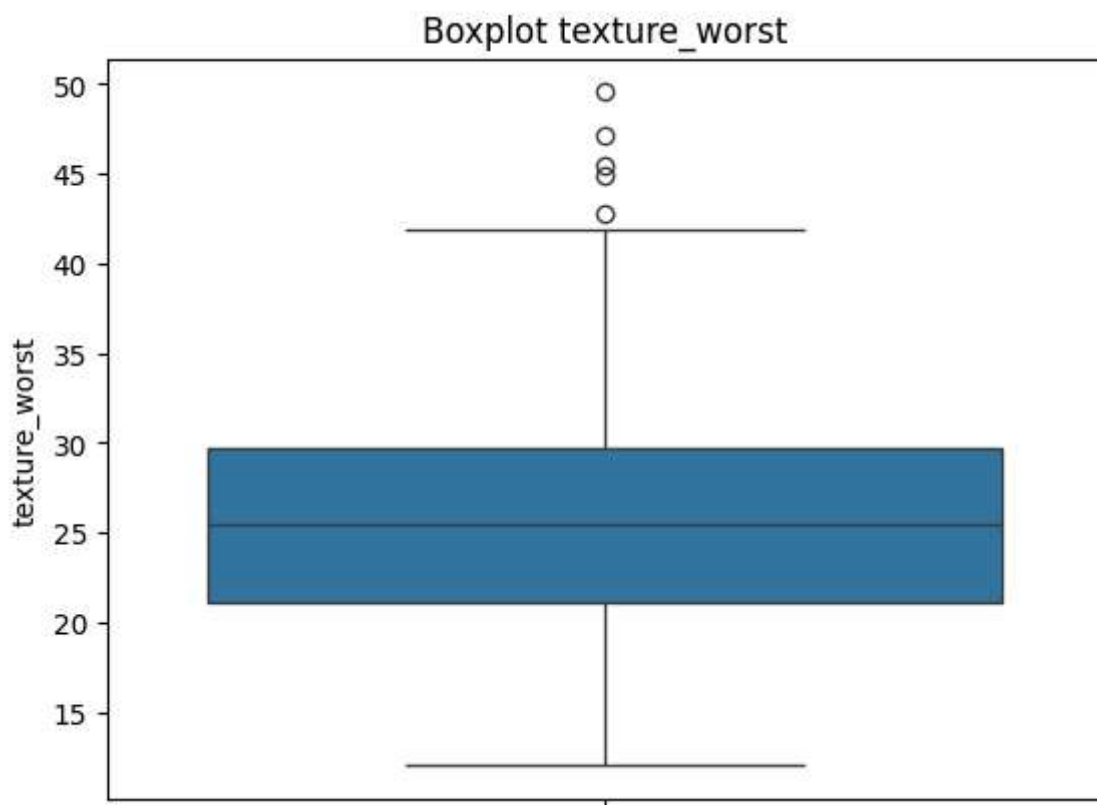
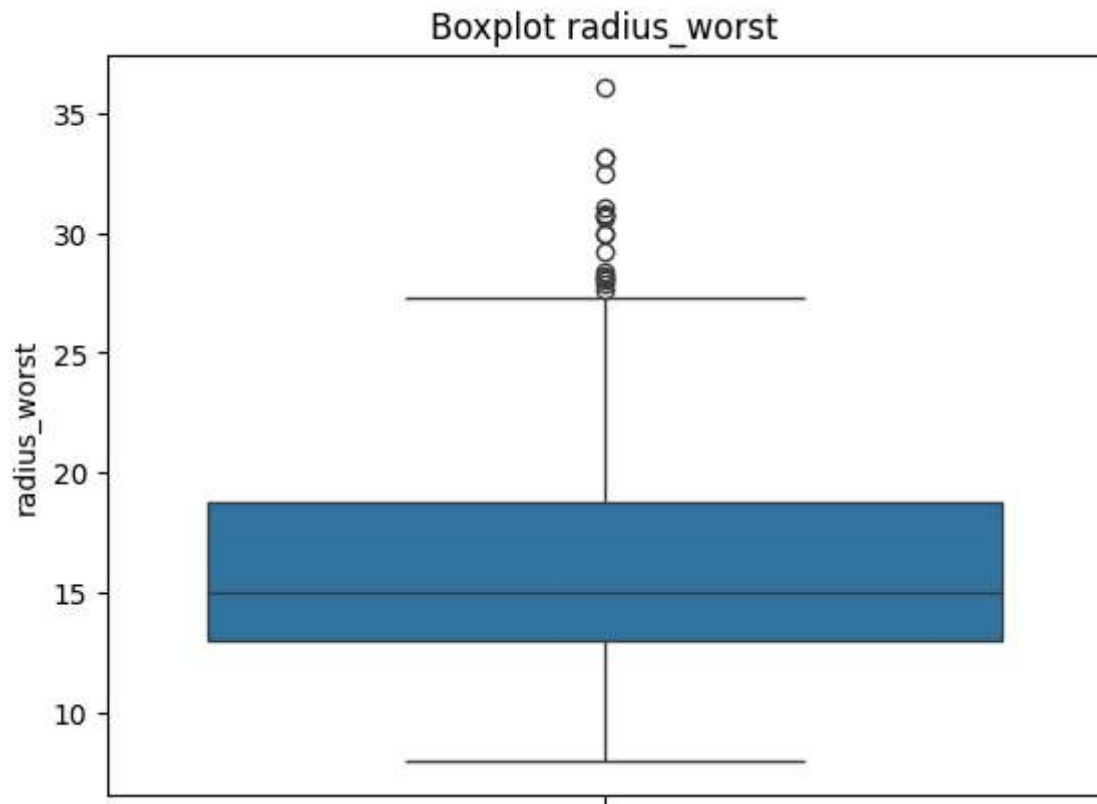


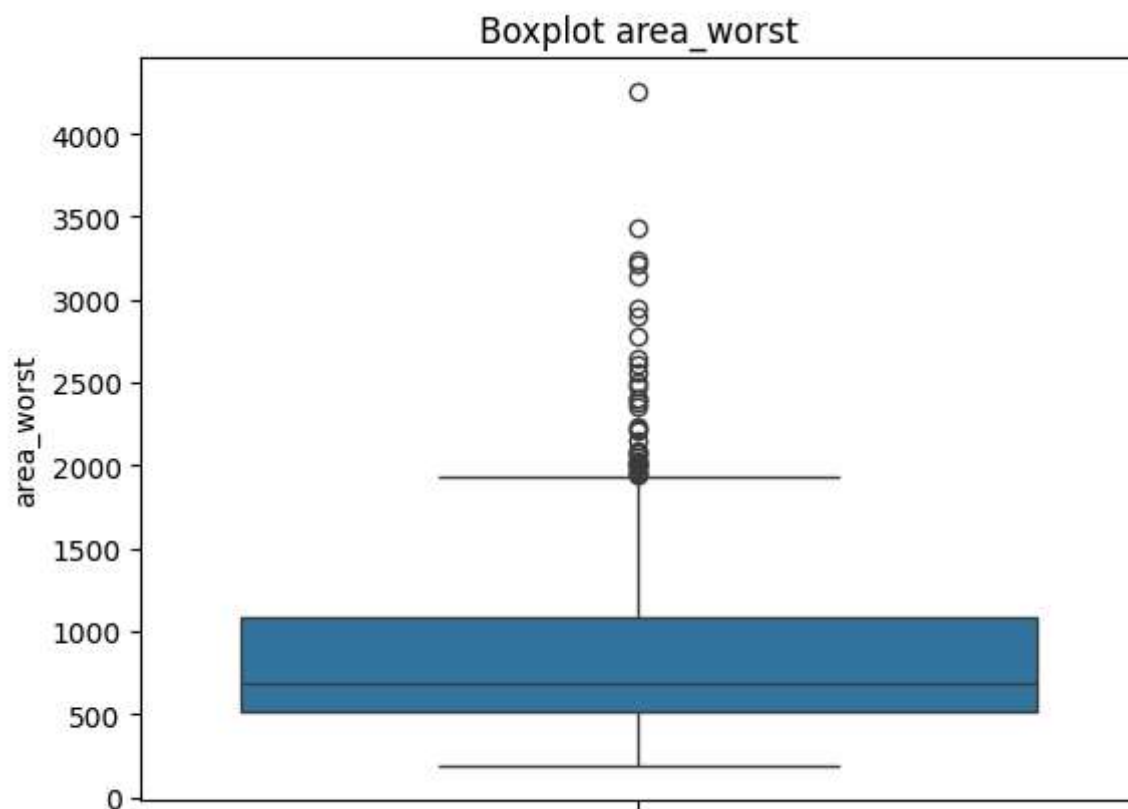
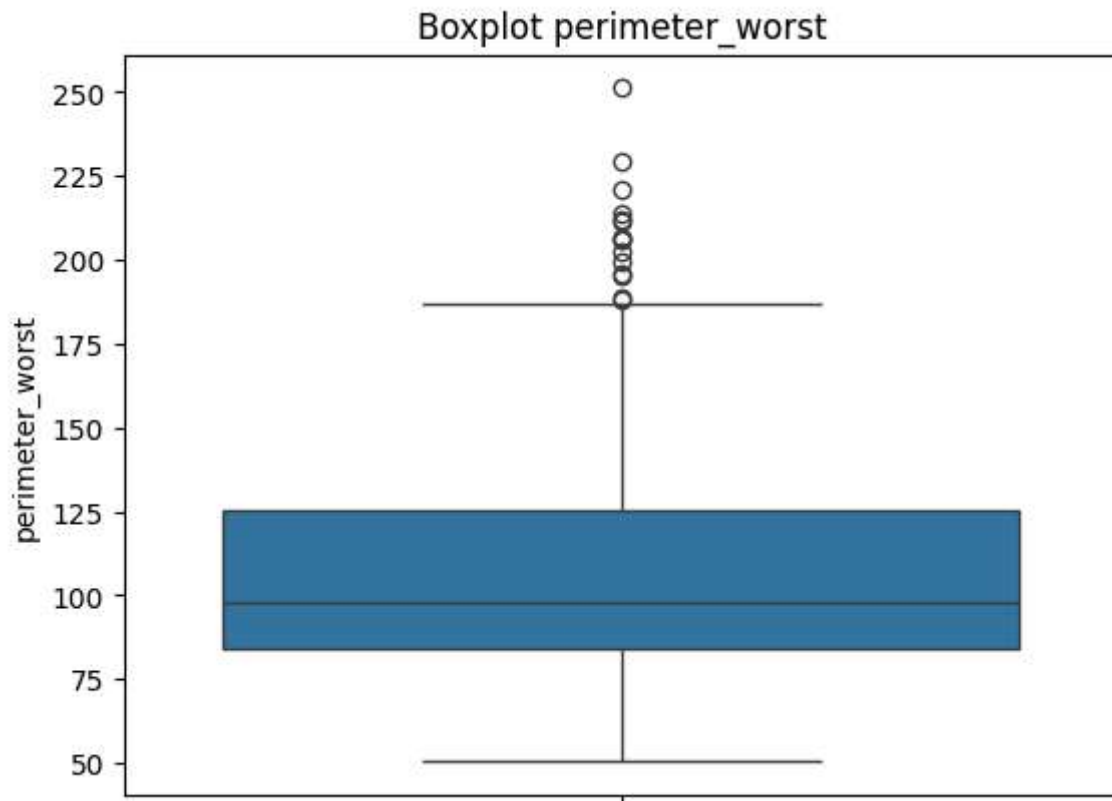


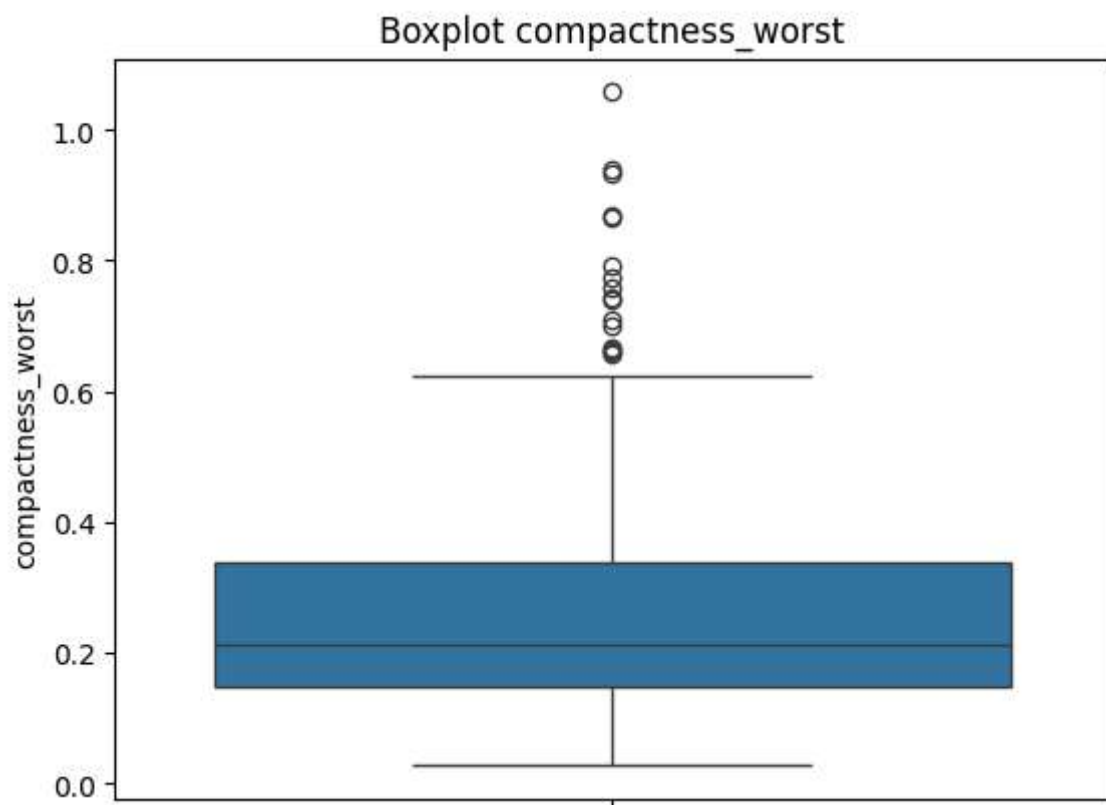
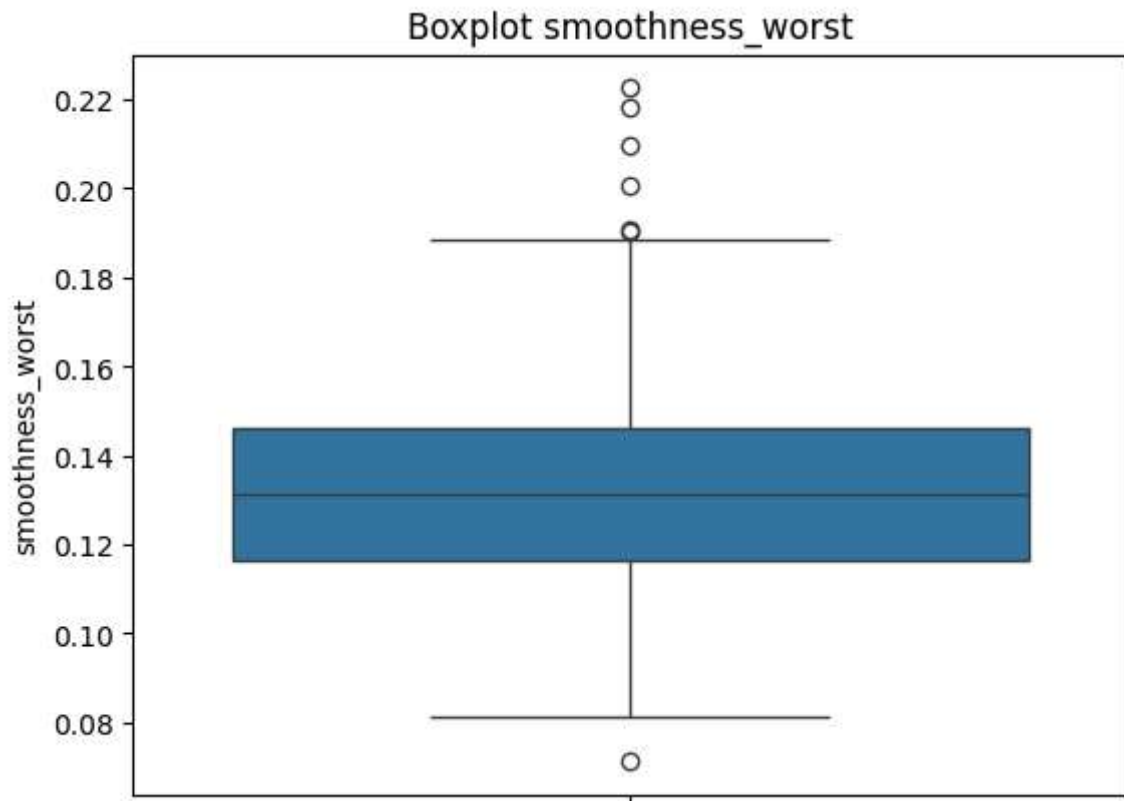


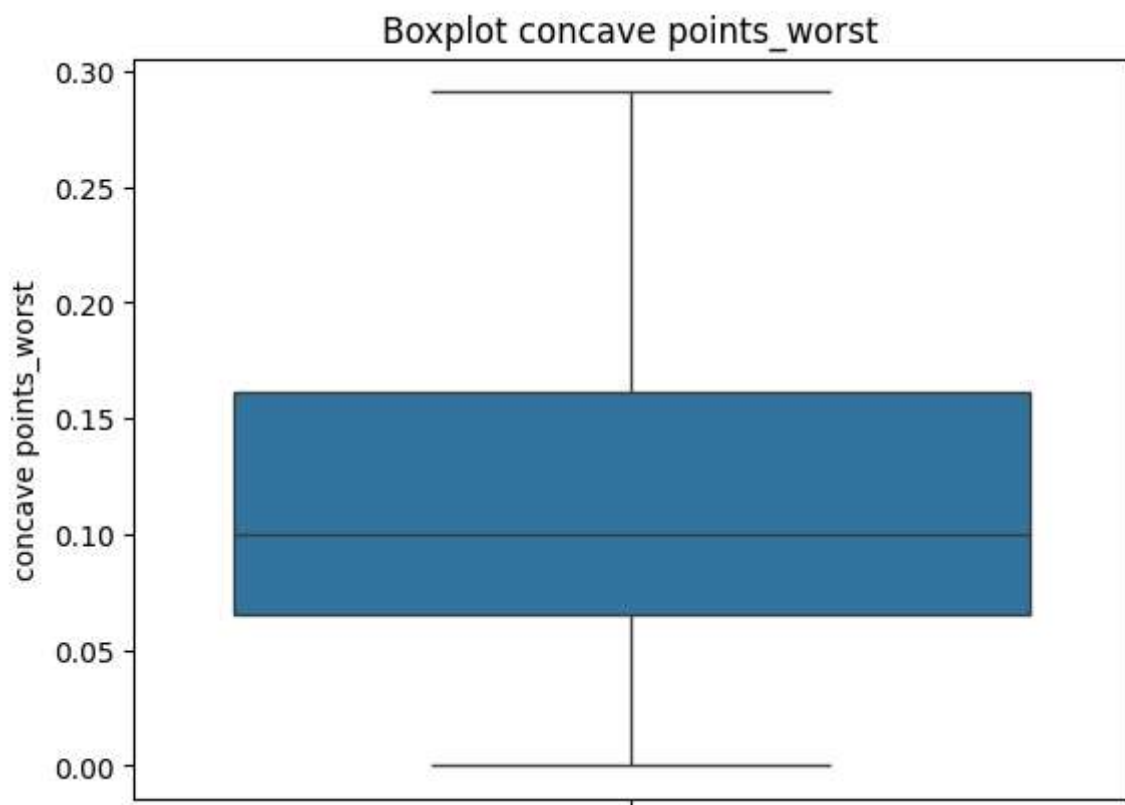
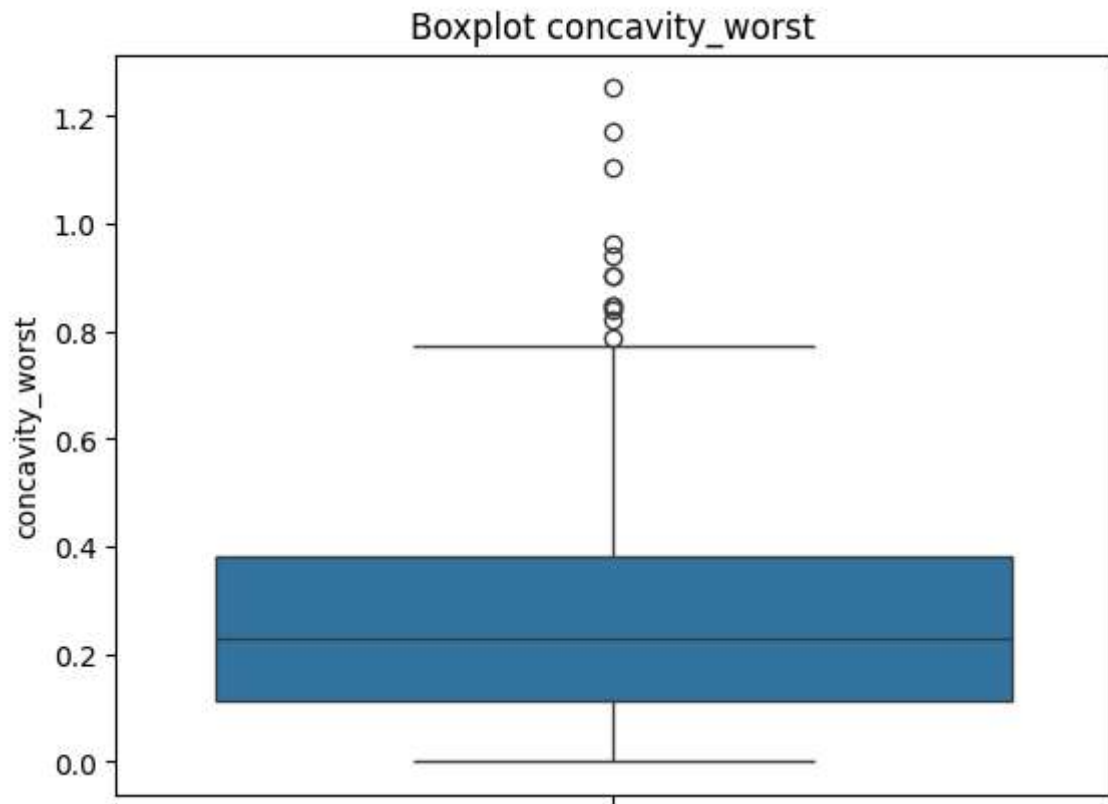


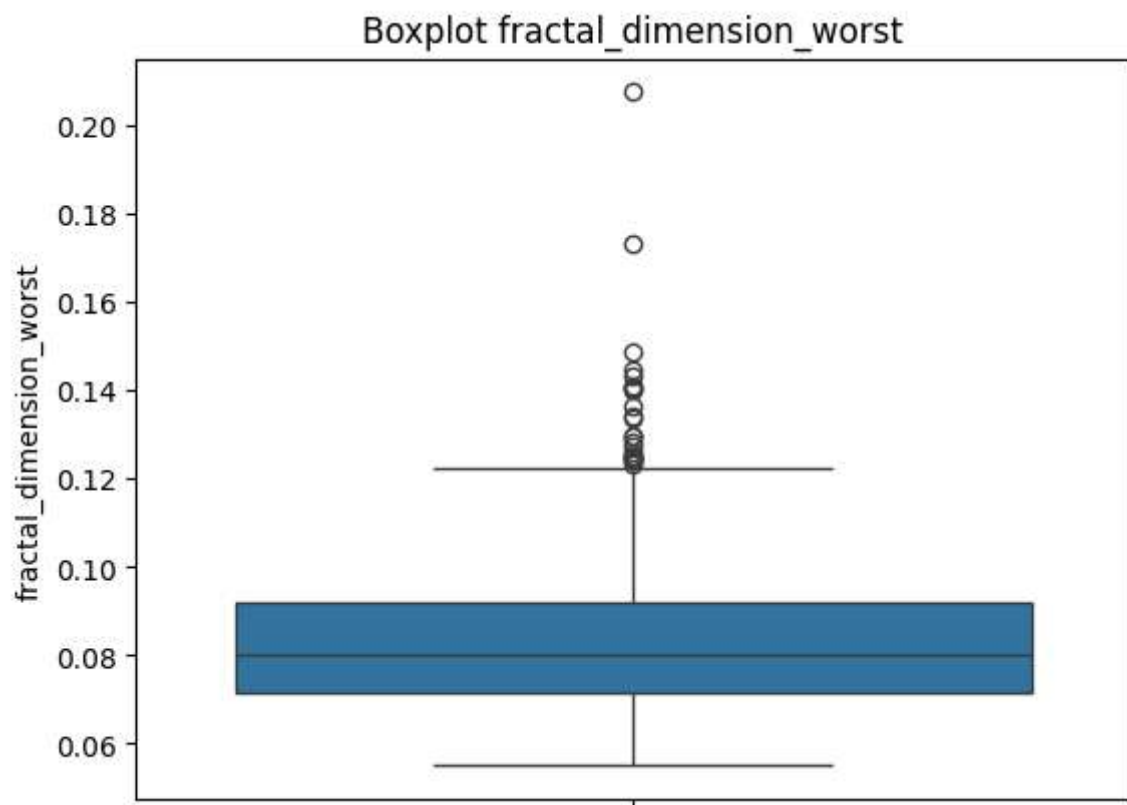
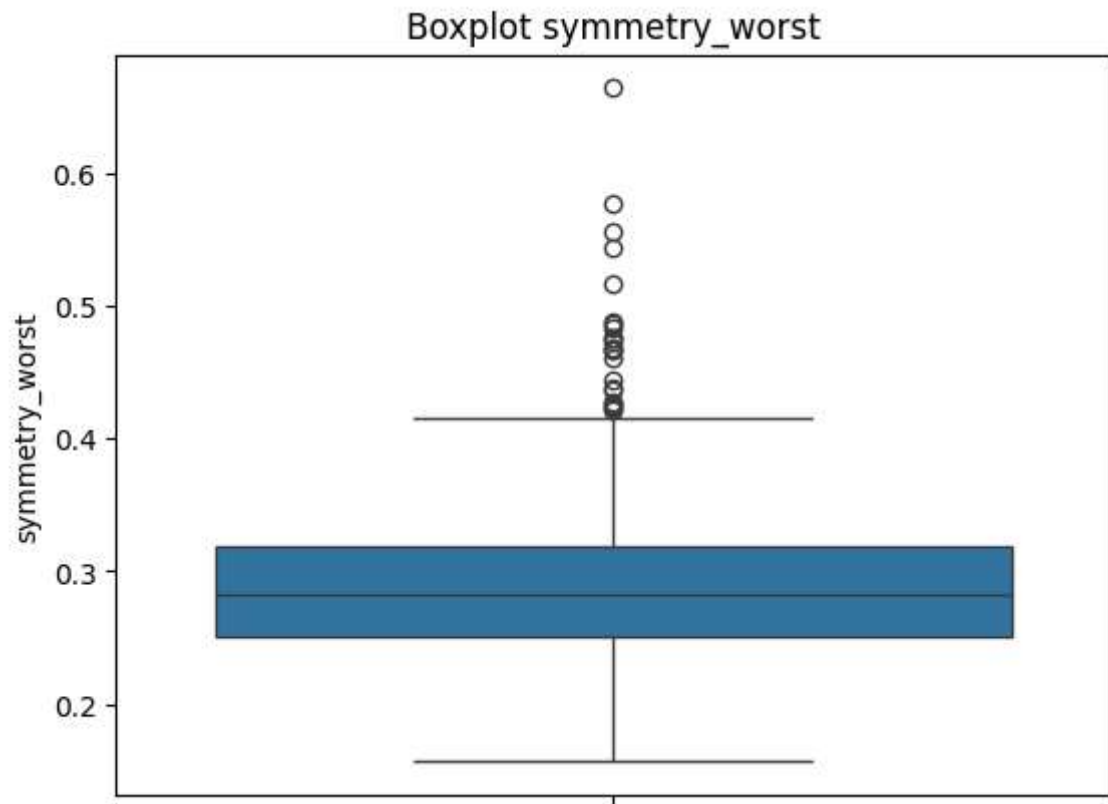


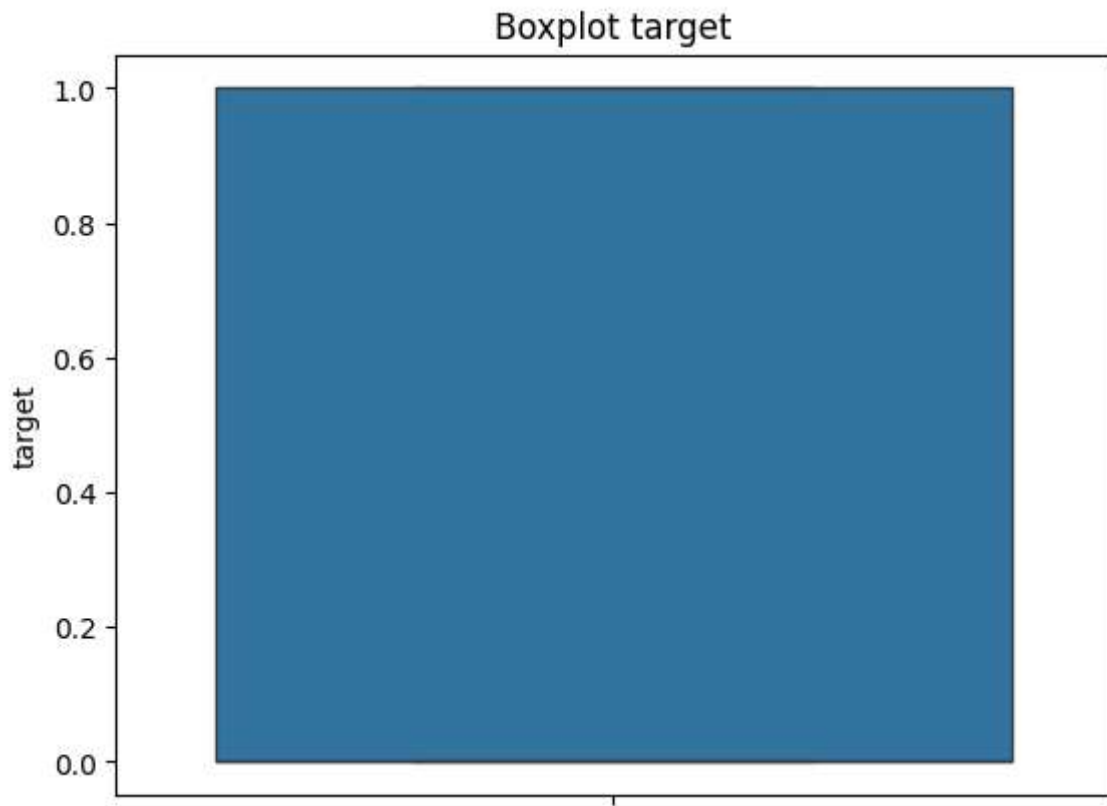




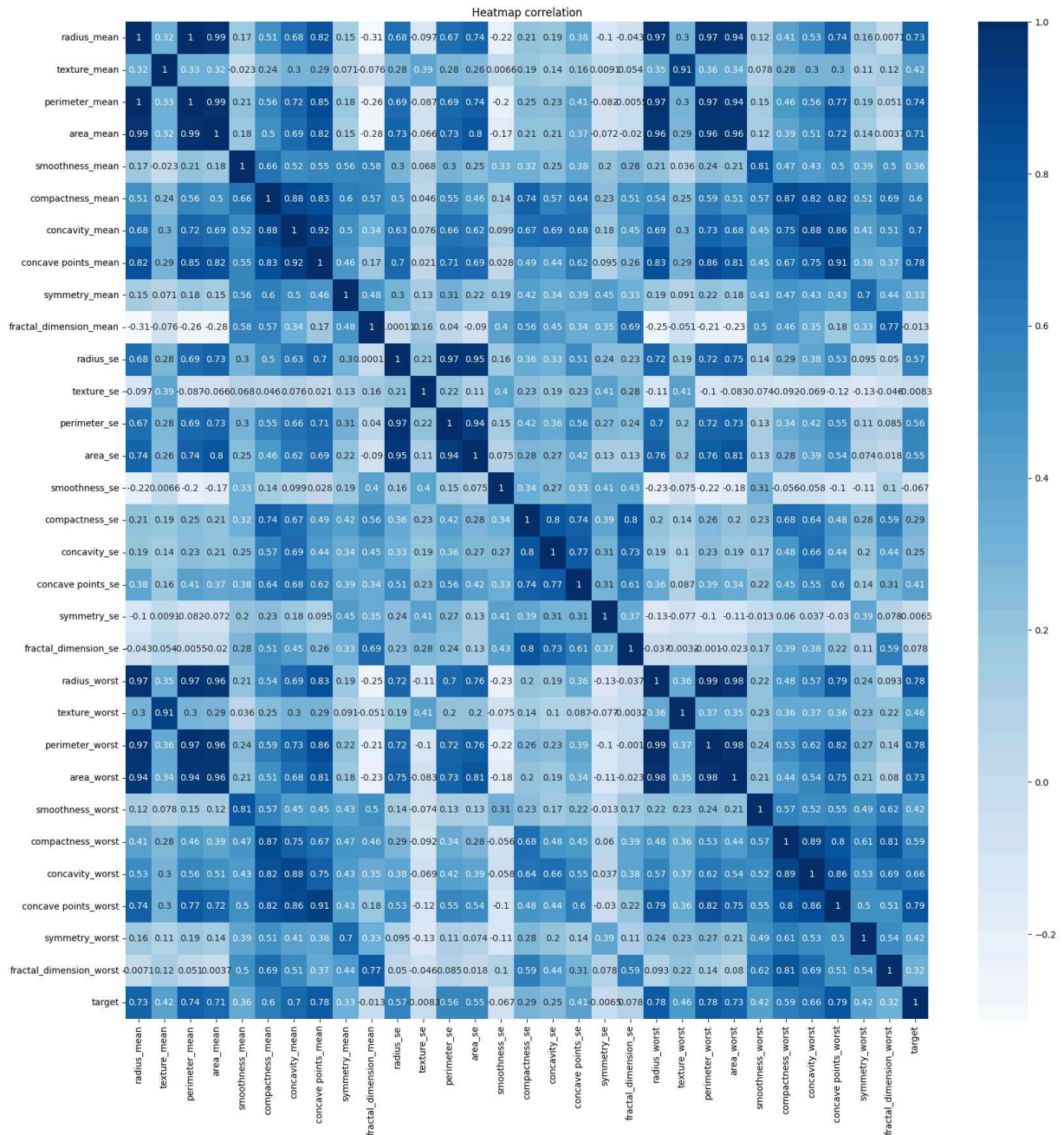








```
In [50]: # correlation matrix
corr_matrix = data.corr()
plt.figure(figsize=(20,20))
sns.heatmap(corr_matrix, annot=True, cmap='Blues')
plt.title("Heatmap correlation")
plt.show()
```



Inference from EDA & Data Visualization: No missing Values All are continuous numerical values except for Target column Mean is slightly more than the median for most of the features. So it is right skewed. Slight imbalance in the dataset (Benign(0) cases are more than Malignant(1) cases Mean of most features are clearly larger for Malignant cases compared to the benign cases (Groupby) Most of the features have Outliers Correlation Matrix reveal that most of the features are highly correlated. So we can remove certain features during Feature Selection

In []: